🚀 Introducción al Desarrollo Web

Introducción

¿Qué hacen realmente los desarrolladores web? En pocas palabras: construyen y mantienen sitios y aplicaciones web.

Los desarrolladores web suelen trabajar para clientes que buscan llevar su producto o servicio al entorno digital. El trabajo se enfoca habitualmente en proyectos e implica colaborar en equipo para transformar las necesidades del cliente en el producto final. El cliente puede ser una empresa de tecnología, una organización o una entidad gubernamental. El trabajo puede abarcar el desarrollo **front-end**, **back-end** o **full-stack**.

El desarrollo web puede ser una excelente profesión para ti si disfrutas resolver problemas lógicos, construir herramientas útiles y experimentar con nuevas tecnologías. Los desarrolladores web tienen una alta demanda laboral, disfrutan en general de un buen equilibrio entre la vida personal y laboral, y perciben salarios competitivos. Te recomendamos investigar el mercado en tu región para conocer las oportunidades locales de empleo.

Resumen de la Lección

En esta sección cubriremos una visión general de los temas que dominarás a lo largo de esta clase:

- Los diferentes tipos de desarrolladores web.
- Los diferentes tipos de clientes y empleadores en la industria.
- Las herramientas clave que utilizarás a diario.

Tipos de Desarrolladores Web

Mencionamos que el desarrollo web se divide en **front-end**, **back-end** y **full-stack**. ¿Qué significan exactamente estos términos?

- **Front-End:** Es todo lo que ves e interactúas en el sitio web a través de tu navegador, incluyendo la presentación del contenido y los elementos de la interfaz de usuario (como la barra de navegación, botones y formularios). Los desarrolladores front-end utilizan **HTML, CSS, JavaScript** y sus *frameworks* modernos para garantizar que la información se presente de forma atractiva y que el usuario tenga una experiencia impecable.
- **Back-End:** Se refiere al motor o "las entrañas" de la aplicación, las cuales se ejecutan en el servidor. El back-end procesa, almacena y sirve los datos necesarios para que el front-end funcione correctamente. Este proceso puede volverse altamente complejo cuando un sitio maneja millones de usuarios. Los desarrolladores back-end emplean lenguajes de programación como **Java, Python, Ruby, PHP o JavaScript (Node.js)** para gestionar la lógica y las bases de datos.
- **Full-Stack:** Son los desarrolladores que se desenvuelven con soltura tanto en el front-end como en el back-end. En **DevToHack Academy**, nos enfocamos en enseñarte desarrollo **Full-Stack**, cubriendo cada pilar del proceso de creación web.

Tipos de Carreras y Salidas Laborales

Ahora que conoces las áreas técnicas, analicemos los entornos de trabajo y los tipos de empleadores con los que te encontrarás:

- **Grandes Empresas Tecnológicas (Big Tech):** Compañías como Google, Meta o Amazon tienen procesos de selección exigentes. Si cumples con sus expectativas, ofrecen excelentes salarios, beneficios de alto nivel y gran proyección profesional.
- **Startups:** Son entornos dinámicos y de alto ritmo. Para un desarrollador *junior*, puede sentirse como una "prueba de fuego" por la velocidad a la que se construye. Suelen ofrecer salarios de entrada más ajustados o jornadas intensas, pero lo compensan con participación accionaria (*equity*) y aprendizaje acelerado.
- **Freelancer (Trabajador Independiente):** Te permite fijar tus tarifas por hora y tener la libertad de gestionar tu tiempo e idear tus propios productos. Sin embargo, serás responsable de captar tus propios clientes (lo que reduce el tiempo de código), gestionar cobros y asumir la responsabilidad total del negocio. Requiere grandes habilidades interpersonales y de negociación.
- **Consultoría Web:** Trabajando para una firma de consultoría cedes parte del potencial de ingresos del *freelance*, pero te concentras 100% en escribir código sin preocuparte por la búsqueda de clientes. Ofrece buen balance vida-trabajo y excelente remuneración.
- **Empresas Tradicionales:** Grandes empresas consolidadas de otros sectores (banca, retail, industria) necesitan constantemente desarrolladores web. Ofrecen

estabilidad, buenos salarios y beneficios, aunque sus procesos tecnológicos suelen avanzar a un ritmo más conservador.

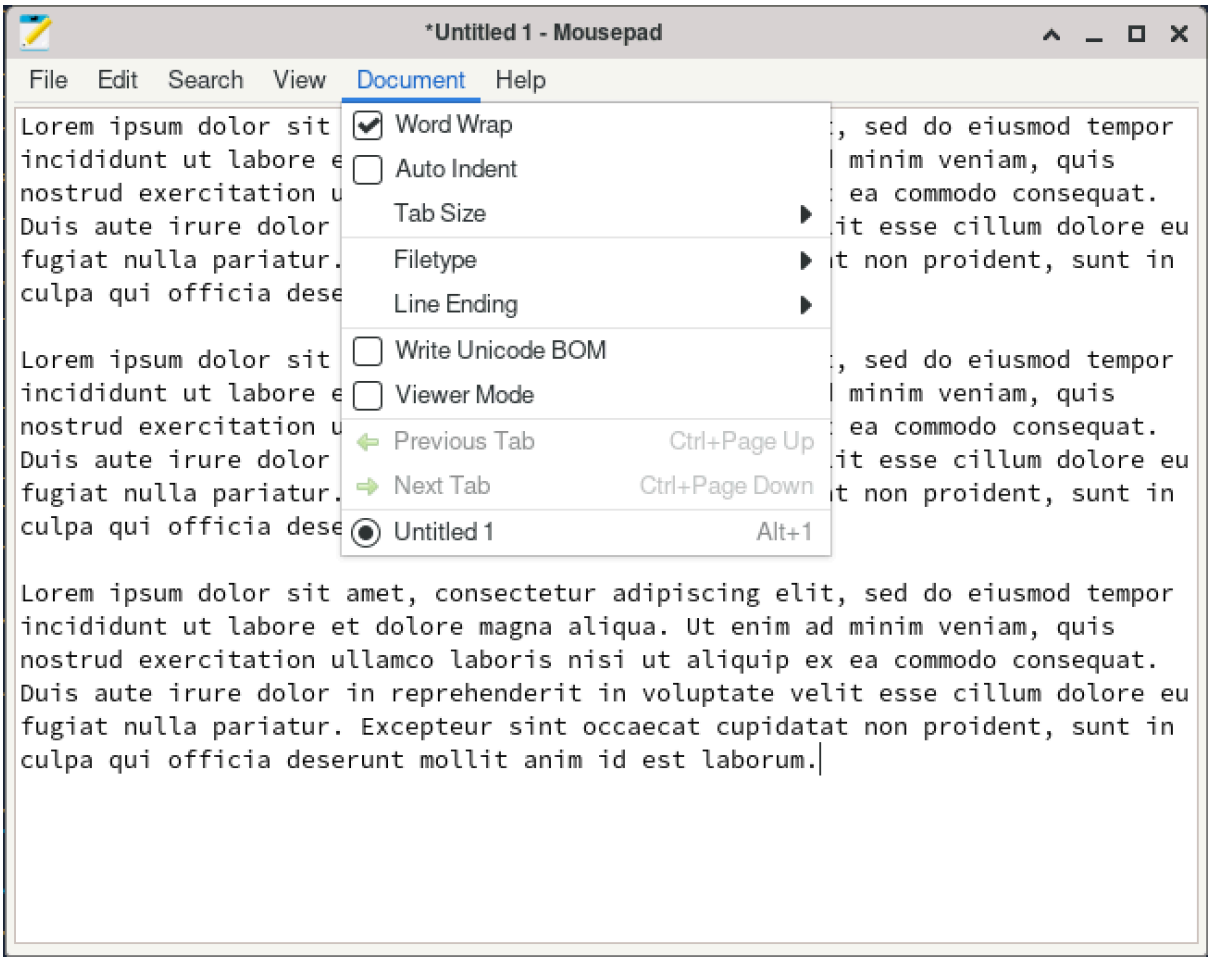
Herramientas de Trabajo (*Tools of the Trade*)

Estas son las herramientas fundamentales que utilizarás continuamente en tu día a día como desarrollador. Si aún no las conoces a fondo, no te preocupes, las dominaremos a lo largo del curso:

1. 💻 **Computadora:** Tu estación de trabajo principal.
2. 🔍 **Motor de Búsqueda:** Herramientas como Google o DuckDuckGo para investigar documentación y resolver errores.
3. 📝 **Editor de Texto / IDE:** Entornos como VS Code para escribir y estructurar tu código.
4. 🖥️ **Interfaz de Línea de Comandos (CLI / Terminal):** Para interactuar directamente con el sistema operativo mediante comandos.
5. 🌐 **Stack Overflow & Comunidades:** Plataformas clave para consultar dudas y soluciones a problemas técnicos reales.
6. 🔀 **Git:** El sistema de control de versiones estándar de la industria para rastrear cambios en tu código.
7. 🐙 **GitHub:** Plataforma en la nube para alojar tus repositorios, colaborar en equipo y mostrar tu portafolio al mundo.

Editor de texto

Un editor de texto es un software interactivo que permite a un usuario editar texto plano, como el Bloc de notas (Notepad).



Editores como Mousepad, mostrado aquí, a menudo se incluyen con un sistema operativo como una aplicación auxiliar (*helper application*) para la edición de archivos de texto.

Al igual que con cualquier software, un editor de texto se puede instalar en un sistema, pero a menudo se incluye un editor de texto relativamente simple en la instalación predeterminada de un sistema operativo (SO), ya que editar archivos de texto es una necesidad relativamente básica para cualquier sistema y debido a que un editor de texto simple se puede proporcionar a un costo relativamente bajo.

Dado que el código fuente es texto, cualquier editor de texto se puede utilizar para editar código, pero un editor de código fuente está diseñado con características específicamente destinadas a la edición de código. Algunos proporcionan integración con herramientas de desarrollo de software y ofrecen un entorno de depuración.

Texto plano y texto enriquecido

Existen diferencias fundamentales entre el **texto plano** (creado y modificado mediante editores de texto) y el **texto enriquecido** (como el producido por procesadores de palabras o programas de maquetación).

El **texto plano** se compone exclusivamente de representaciones de caracteres. Cada uno de estos caracteres se almacena como una secuencia de bytes (ya sea de longitud fija o variable, de uno a cuatro bytes) siguiendo estándares de codificación como ASCII, ISO/IEC 2022, Shift JIS, UTF-8 o UTF-16. Estos estándares no solo definen los caracteres visibles, sino también aquellos que controlan la estructura y el flujo del texto (como espacios, saltos de línea y saltos de página). El texto plano no almacena información adicional sobre sí mismo, ni siquiera el tipo de codificación empleado. Se guarda en archivos de texto (aunque estos no alberguen únicamente texto plano). Históricamente, y por convención actual, suele mostrarse con tipografías monoespaciadas, lo que permite alinear texto o formatear en columnas usando espacios en blanco.

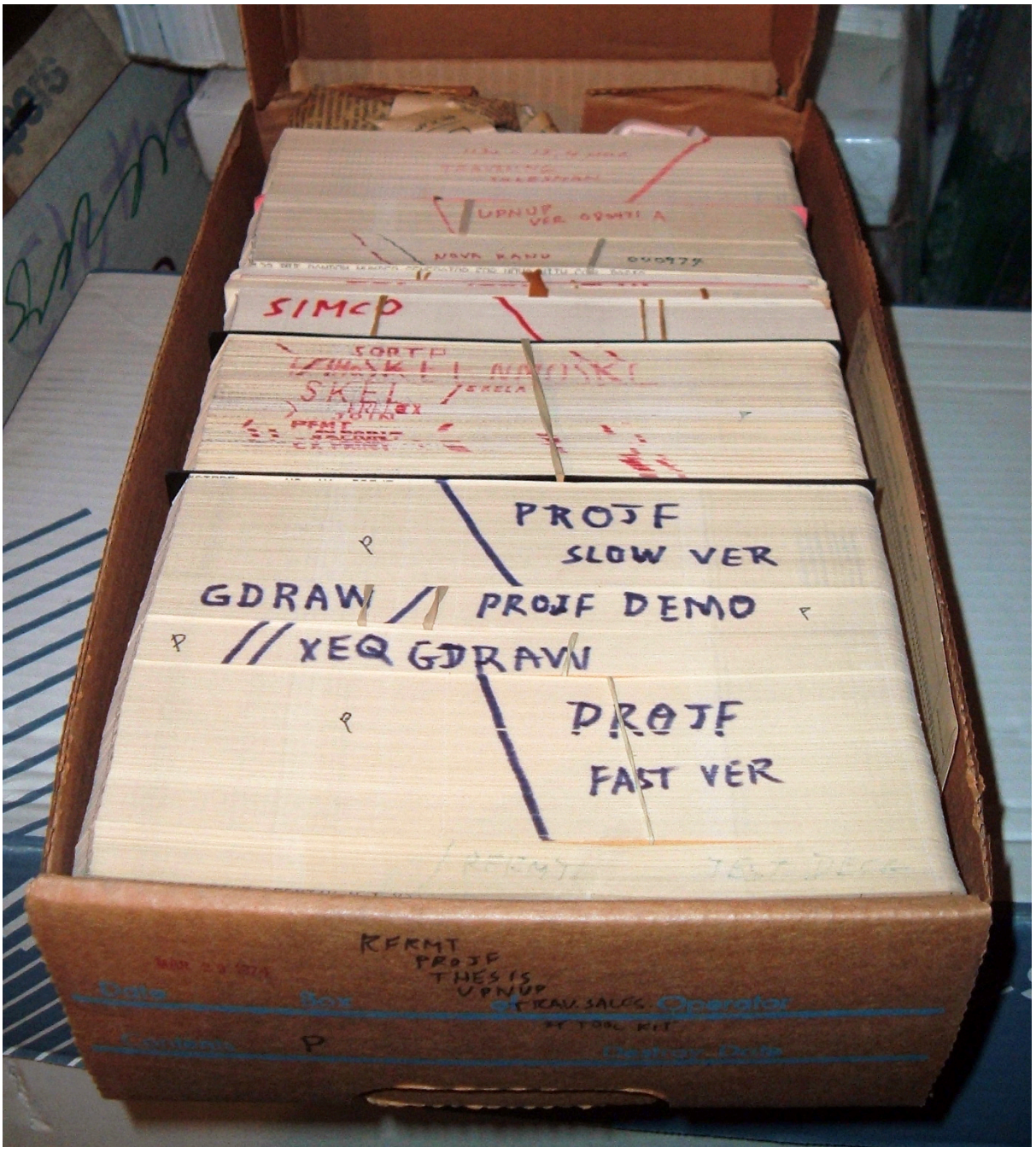
Por su parte, el **texto enriquecido** incluye metadatos y diversos datos de formato. Esto abarca aspectos tipográficos (fuente, tamaño, grosor y estilo), de párrafo (sangrías, alineación, espaciado entre líneas o párrafos) y parámetros de página (márgenes, dimensiones y orientación de lectura). Su estructura puede ser muy compleja y guardarse en formatos binarios (como `.doc`), en archivos estructurados mediante lenguajes de marcado (como `.rtf` o `.html`), o en formatos híbridos (como `.docx` u Office Open XML).

Los editores de texto están diseñados específicamente para abrir y guardar archivos que contengan texto plano o cualquier contenido que pueda interpretarse como tal, incluyendo el código fuente de marcado de texto enriquecido o de otros formatos vectoriales y gráficos (como `.svg`).

Historia y origen de los editores de texto

La era previa a los editores interactivos

Antes de que existieran los editores de texto, el texto en las computadoras se perforaba en tarjetas mediante máquinas perforadoras (*keypunch*). Las cajas físicas de estas delgadas tarjetas de cartón se introducían luego en un lector de tarjetas. Los archivos de imagen de tarjeta en cinta magnética, tambor y disco creados a partir de esos paquetes de tarjetas a menudo no tenían caracteres de separación de línea en absoluto, y asumían registros de longitud fija de 80 o 90 caracteres. Una alternativa a las tarjetas era la cinta perforada. Esta podía ser creada por algunos teleimpresores (como el Teletype), los cuales utilizaban caracteres especiales para indicar el final de los registros. Algunos sistemas operativos primitivos incluían editores de texto por lotes (*batch*), ya fuera integrados con procesadores de lenguaje o como programas de utilidad independientes; un ejemplo temprano fue la capacidad de editar archivos fuente SQUOZE para SCAT en el sistema operativo SHARE.



Una caja de tarjetas perforadas con varios mazos de programas (*program decks*).

Los primeros editores de texto interactivos fueron "editores de línea" orientados a teletipos: terminales con estilo de teleimpresor o máquina de escribir que imprimían mecánicamente tanto la entrada como la salida en el mismo rollo de papel continuo, sin una pantalla iluminada.

Los comandos (a menudo una sola pulsación de tecla) realizaban ediciones en un archivo en un punto de inserción del cual el operador debía llevar el control, llamado "cursor". Las ediciones se verificaban escribiendo un comando para imprimir una pequeña sección del archivo y, de forma periódica, imprimiendo el archivo completo.

En algunos editores de línea, el cursor se podía mover mediante comandos que especificaban el número de línea en el archivo, cadenas de texto (contexto) a buscar y, eventualmente, expresiones regulares. Los editores de línea representaron una mejora sustancial con respecto a las tarjetas perforadas. Algunos de ellos podían utilizarse mediante tarjetas perforadas; los comandos de edición se podían tomar de un paquete de tarjetas y aplicarse a un archivo específico. Varios editores de línea comunes admitían un modo de "verificación" (*verify*) en el que los comandos de cambio mostraban las líneas modificadas.

Un ejemplo de configuración, hacia 1975, era un Teletype Modelo 33 como consola de una PDP-11 ejecutando Version 6 Unix, manipulando texto con `ed`, el editor de texto estándar de UNIX.

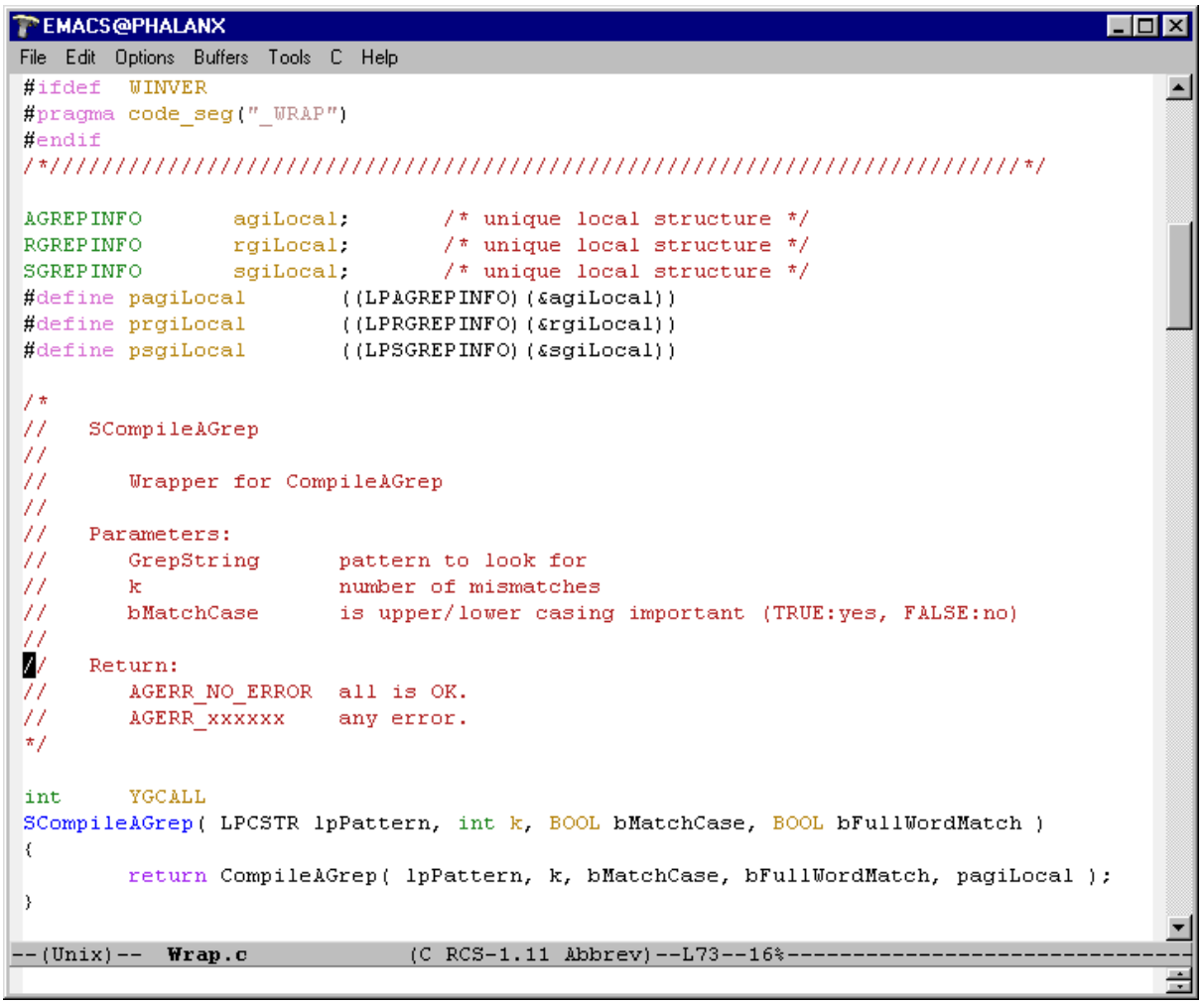
Cuando aparecieron las terminales de computadora con pantallas de video, los editores de texto basados en pantalla (a veces llamados simplemente "editores de pantalla") se volvieron comunes. Uno de los primeros editores a pantalla completa fue O26, escrito para la consola del operador de las computadoras de la serie CDC 6000 en 1967. Otro editor primitivo a pantalla completa fue `vi`. Escrito en la década de 1970, sigue siendo un editor estándar en los sistemas operativos Unix y Linux. También desarrollado en la década de 1970 estuvo el *UCSD Pascal Screen Oriented Editor*, optimizado tanto para código fuente con sangría como para texto general. Emacs, uno de los primeros proyectos de software libre y de código abierto, es otro editor temprano a pantalla completa o en tiempo real, el cual fue portado a muchos sistemas. El Commodore PET de 1977 fue la primera computadora de gran consumo en incluir un editor a pantalla completa. La facilidad de uso y la velocidad de un editor a pantalla completa (en comparación con los editores basados en líneas) motivaron muchas de las primeras compras de terminales de video.

La estructura de datos central en un editor de texto es la que gestiona la cadena (secuencia de caracteres) o lista de registros que representa el estado actual del archivo que se está editando. Mientras que la primera podía almacenarse en un único arreglo consecutivo y largo de caracteres, el deseo de contar con editores de texto que pudieran insertar texto, eliminar texto y deshacer/rehacer ediciones previas con mayor rapidez llevó al desarrollo de estructuras de datos secuenciales más complejas. Un editor de texto típico utiliza un *gap buffer*, una lista enlazada de líneas (como en PaperClip), una tabla de piezas (*piece table*) o una estructura *rope* como su estructura de datos secuencial.

Tipología

Algunos editores de texto son pequeños y sencillos, mientras que otros ofrecen funciones amplias y complejas. Por ejemplo, los sistemas operativos Unix y similares a Unix cuentan con el editor `pico` (o una variante), pero muchos también incluyen los editores `vi` y `Emacs`. Los sistemas Microsoft Windows vienen con el sencillo Bloc de notas (*Notepad*), aunque muchas personas —especialmente los programadores— prefieren otros editores con más

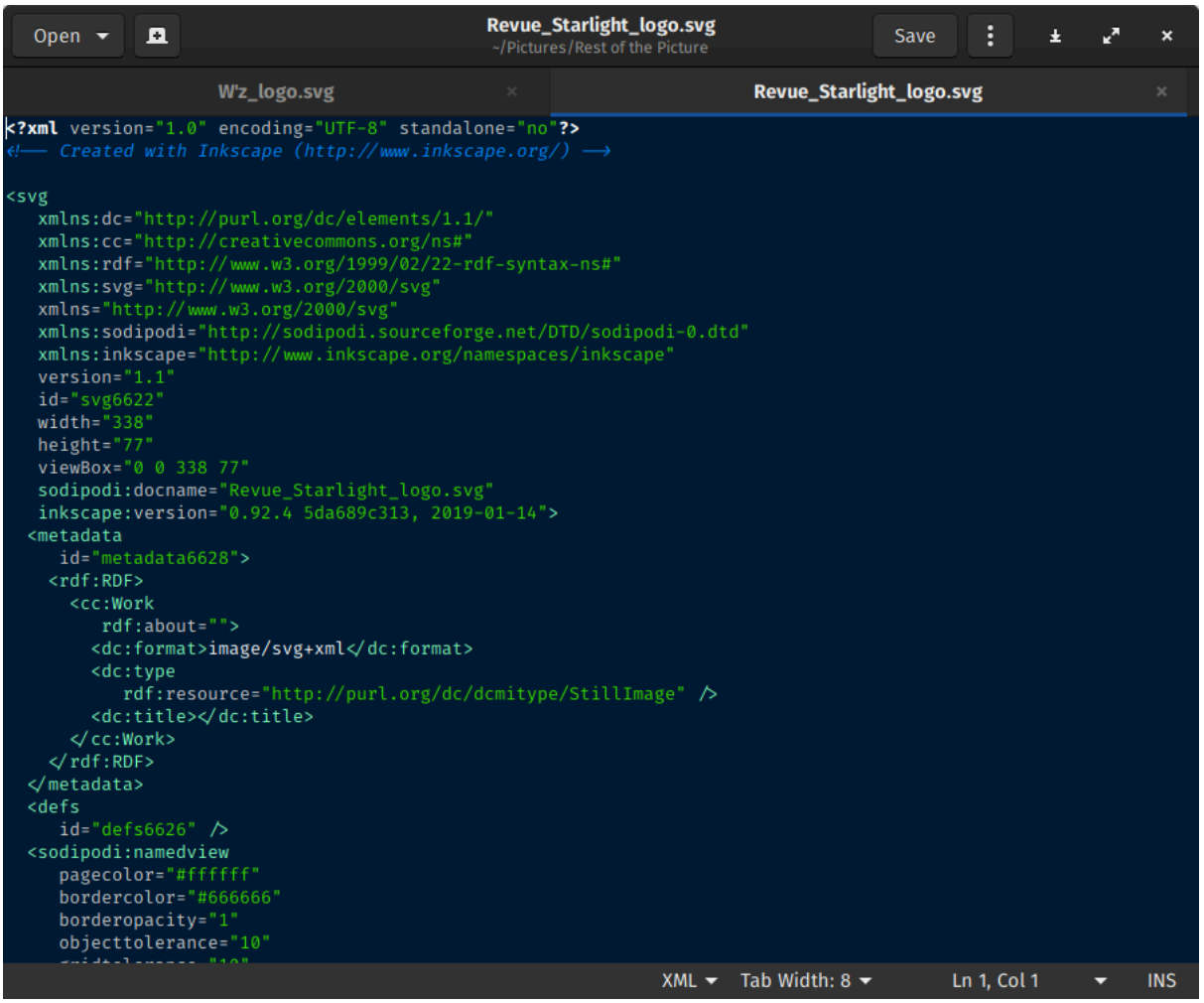
características. En el clásico Mac OS de Apple Macintosh existía el nativo *TeachText*, reemplazado posteriormente por *SimpleText* en 1994, el cual fue sustituido en Mac OS X por *TextEdit*, que combina características de un editor de texto con aquellas típicas de un procesador de texto, tales como reglas, márgenes y selección de múltiples fuentes tipográficas. Estas funciones no están disponibles simultáneamente, sino que deben alternarse por comando del usuario o mediante la determinación automática del tipo de archivo por parte del programa.



Emacs, un editor de texto popular entre los programadores, ejecutándose en Microsoft Windows.

La mayoría de los procesadores de texto pueden leer y escribir archivos en formato de texto plano, lo que les permite abrir archivos guardados desde editores de texto. Sin embargo, guardar estos archivos desde un procesador de texto requiere garantizar que el archivo se escriba en formato de texto plano y que ninguna configuración de codificación de texto o marca de orden de bytes (BOM) dificulte el uso previsto del archivo. Los procesadores de texto que no son WYSIWYG, como *WordStar*, se adaptan más fácilmente para funcionar como editores de texto y, de hecho, se utilizaban comúnmente como tales durante la década de 1980. El formato de archivo predeterminado de estos procesadores de texto a menudo se asemeja a un lenguaje de marcado, siendo el formato básico el texto plano y logrando el formato visual mediante secuencias de escape o caracteres de control no

imprimibles. Los procesadores de texto posteriores, como Microsoft Word, guardan sus archivos en un formato binario y casi nunca se utilizan para editar archivos de texto plano.



gedit es un editor de texto que anteriormente se distribuía con GNOME.

Algunos editores de texto pueden editar archivos inusualmente grandes, como archivos de registro (*log files*) o una base de datos entera ubicada en un solo archivo. Los editores de texto más sencillos pueden simplemente leer los archivos cargándolos directamente en la memoria principal de la computadora. Con archivos de mayor tamaño, este puede ser un proceso lento y es posible que el archivo completo no quepa en memoria. Algunos editores de texto no permiten que el usuario comience a editar hasta que esta lectura se haya completado. El rendimiento de edición también suele verse afectado en editores no especializados, donde el editor puede tardar segundos o incluso minutos en responder a las pulsaciones de teclas o a los comandos de navegación. Los editores especializados cuentan con optimizaciones como almacenar en memoria únicamente la porción visible de los archivos grandes, mejorando así el rendimiento de edición.

Algunos editores son programables, lo que significa, por ejemplo, que pueden personalizarse para usos específicos. Con un editor programable es fácil automatizar tareas repetitivas, añadir nueva funcionalidad o incluso implementar una nueva aplicación dentro del propio marco del editor. Un motivo común para la personalización es lograr que un editor

de texto utilice los comandos de otro editor con el que el usuario esté más familiarizado, o duplicar funcionalidades faltantes de las que el usuario ha llegado a depender. Los desarrolladores de software a menudo utilizan personalizaciones de editores adaptadas al lenguaje de programación o al entorno de desarrollo en el que están trabajando. La programabilidad de algunos editores de texto se limita a mejorar la funcionalidad principal de edición del programa, pero Emacs se puede extender mucho más allá de la edición de archivos de texto: para navegación web, lectura de correo electrónico, chat en línea, gestión de archivos o juegos, y a menudo se le considera como un entorno de ejecución Lisp con una interfaz de usuario basada en texto. Emacs incluso se puede programar para emular a Vi, su rival en las tradicionales guerras de editores de la cultura Unix.

Una categoría destacada dentro de los editores programables utiliza **REXX** como lenguaje para la creación de *scripts*. Estos entornos, conocidos como **"editores ortodoxos"**, incorporan una línea de comandos integrada donde es posible introducir órdenes y macros, así como un espacio directo en las líneas de texto para ejecutar comandos de línea. La gran mayoría de estas herramientas deriva de los sistemas **ISPF/PDF EDIT** o de **XEDIT** (el editor insigne de IBM para entornos VM/SP hasta z/VM), contando entre sus exponentes a programas como **THE, KEDIT, X2, Uni-edit** y **SEDIT**.

Por otro lado, cuando un editor de texto se diseña o personaliza para un flujo de trabajo específico, este adquiere la capacidad de identificar el contenido que el usuario modifica para asistirlo de manera activa. Entre sus funciones habituales se encuentran el autocompletado de términos de programación y la muestra de información contextual o documentación en ventanas emergentes.

Para facilitar la lectura y escritura de código fuente, la mayoría de los editores para desarrolladores incluye **resaltado de sintaxis** y **sangría automática**. Asimismo, permiten seleccionar directamente nombres de funciones, variables o archivos incluidos para saltar a su definición original. Para regresar al punto de origen, el editor suele almacenar la ubicación inicial del cursor o desplegar la información en un búfer temporal. Aunque algunos editores implementan esta navegación de forma nativa, es muy común recurrir a utilidades auxiliares como **ctags** para indexar y localizar dichas definiciones.

Características típicas

Cortar, copiar y pegar La mayoría de los editores de texto proporcionan métodos para duplicar y mover texto dentro del archivo o entre archivos.

Integración con el portapapeles Cortar, copiar y pegar a menudo están integrados con el portapapeles del sistema.

Buscar y reemplazar Los editores de texto proporcionan amplias funciones para buscar y reemplazar cadenas de texto, ya sea de forma individual o en grupos de archivos dentro de pestañas abiertas o en una carpeta seleccionada. Los editores avanzados pueden utilizar

expresiones regulares para buscar y editar texto o código. Las características adicionales pueden incluir la distinción opcional de mayúsculas y minúsculas, un historial de términos de búsqueda para un acceso rápido y autocompletado, y el listado de múltiples resultados en un solo lugar.

Deshacer y rehacer (*Undo and redo*)

Forma de deshacer y rehacer ediciones previas. A menudo —especialmente en editores de texto más antiguos— solo se recuerda un nivel de historial de edición y la emisión sucesiva del comando deshacer solo "alternará" el último cambio. Los editores modernos o más complejos suelen proporcionar un historial de múltiples niveles, de modo que la emisión repetida del comando deshacer revertirá el documento a ediciones sucesivamente más antiguas. Un comando rehacer independiente desplazará las ediciones "hacia adelante" hacia los cambios más recientes.

Soporte UTF-8 (*UTF-8 support*)

Capacidad para manejar texto codificado en UTF-8.

Formato básico (*Basic formatting*)

Los editores de texto a menudo proporcionan características de formato básico como el ajuste de línea (*line wrap*) y la sangría automática (*auto-indentation*).

Ir a la línea (*Jump to line*)

Capacidad para saltar a un número de línea especificado.

Funciones avanzadas

Definición de macros o procedimientos (*Macro or procedure definition*)

Capacidad para definir nuevos comandos o características como combinaciones de comandos previos u otras macros, posiblemente con parámetros pasados o con anidamiento de macros.

Perfiles (*Profiles*)

Capacidad para conservar las opciones configuradas por el usuario entre sesiones de edición.

Macros de perfil (*Profile macros*)

Macros con nombres especificados en, por ejemplo, variables de entorno o archivos de perfil, que se ejecutan automáticamente al inicio de una sesión de edición o al abrir un nuevo archivo.

Edición basada en columnas (*Column-based editing*)

La capacidad de alterar o insertar datos en una columna específica, o de desplazar datos hacia columnas determinadas.

Transformación de datos (*Data transformation*)

Leer o fusionar el contenido de otro archivo de texto en el archivo que se está editando actualmente. Algunos editores de texto proporcionan una forma de insertar la salida de un comando enviado al *shell* del sistema operativo. Asimismo, una función de cambio de caja (*case-shifting*) permite convertir texto a minúsculas o mayúsculas.

Filtrado (*Filtering*)

Algunos editores de texto avanzados permiten enviar todo el archivo o secciones del mismo a otra utilidad del sistema y volver a leer el resultado en el archivo, reemplazando las líneas "filtradas". Esto es útil, por ejemplo, para ordenar una serie de líneas alfabética o numéricamente, realizar cálculos matemáticos, aplicar sangría al código fuente, entre otras tareas.

Resaltado de sintaxis (*Syntax highlighting*)

Resalta de forma contextual el código fuente, lenguajes de marcado, archivos de configuración y otros textos que aparezcan en un formato organizado o predecible. Por lo general, los editores permiten a los usuarios personalizar los colores o estilos utilizados para cada elemento del lenguaje. Algunos editores de texto también permiten instalar y utilizar temas para cambiar el aspecto visual (*look and feel*) de toda la interfaz de usuario del editor.

Editores orientados a la sintaxis (*Syntax-oriented editors*)

Algunos editores cuentan con soporte para la sintaxis de uno o más lenguajes y permiten realizar operaciones basadas en unidades sintácticas; por ejemplo, insertar una nueva cláusula `WHEN` dentro de una sentencia `SELECT`.

Extensibilidad (*Extensibility*)

Pueden proporcionar un mecanismo de complementos (*plugins*) o ser programables mediante *scripts*, permitiendo que el usuario personalice el editor.

Navegación del cursor (*Cursor navigation*)

Puede variar entre distintos editores de texto. Por ejemplo, presionar la tecla `Fin` dos veces puede llevar el cursor al final de una línea ajustada (*wrapped line*) después de que la primera pulsación lo haya desplazado al final de la fila de texto visible en pantalla. Las

terminales orientadas a bloques suelen tener teclas dedicadas para el movimiento del cursor, al igual que los teclados de las computadoras personales.

Línea de comandos (*Command line*)

Algunos editores (como ISPF o XEDIT) disponen de un campo dedicado en pantalla para ingresar comandos en lugar de texto. Dependiendo del editor, el usuario puede necesitar usar las teclas de dirección para alternar entre el campo de comandos y el de texto, o el editor puede interpretar teclas de función específicas como solicitudes para cambiar entre ambos.

Comandos de línea (*Line commands*)

(También conocidos como comandos de prefijo o comandos de secuencia)

Algunos editores tratan el archivo como una matriz (*array*) de líneas de texto con números de línea o números de secuencia asociados, y disponen de un campo de número de línea independiente para cada campo de texto. Un comando de línea es una cadena de caracteres que el usuario escribe dentro del campo de número de línea y que el editor reconoce como una orden para operar sobre esa línea específica o bloque de líneas (por ejemplo, `LC` para convertir una línea a minúsculas, o `))3` para desplazar un bloque tres columnas a la derecha). Algunos editores también admiten macros de línea (conocidas como macros de prefijo o macros de secuencia). A pesar del nombre "comando de prefijo", algunos editores permiten que el campo de secuencia aparezca después del campo de texto.

Fuente monoespaciada (*Monospace font*)

Los editores de texto, especialmente los editores de código fuente, suelen utilizar de forma predeterminada una fuente monoespaciada que distingue claramente entre caracteres similares (homóglifos), como los dos puntos y el punto y coma.

Editores especializados

Algunos editores incluyen características especiales y funciones adicionales; por ejemplo:

Editor de código fuente (*Source-code editor*)

Editores de texto con funcionalidad adicional para facilitar la producción de código fuente. Estos suelen contar con resaltado de sintaxis personalizable por el usuario y funciones de navegación de código, así como herramientas de programación o macros de teclado similares a las de un editor HTML.

Editor con plegado (*Folding editor*)

Incluye los llamados "editores ortodoxos" que son derivados de Xedit. Los editores que implementan el plegado sin características específicas para programación suelen denominarse estructuradores (*outliners*, ver más abajo).

Estructurador (*Outliner*)

También llamado editor basado en árbol, ya que combina una vista de árbol esquemática jerárquica con un editor de texto. El plegado puede considerarse una forma especializada de estructuración (*outlining*).

Entorno de desarrollo integrado (*Integrated development environment - IDE*)

Diseñados para gestionar y agilizar grandes proyectos de programación. Por lo general, solo se utilizan para programar, ya que contienen muchas características innecesarias para la edición simple de texto.

Editor HTML (*HTML editor*)

A los creadores del World Wide Web se les ofrece una variedad de editores HTML dedicados a la tarea de crear páginas web. Estos incluyen: Dreamweaver, KompoZer y E Text Editor. Muchos ofrecen la opción de visualizar el trabajo en progreso en un motor de renderizado HTML integrado o en un navegador web estándar. Sin embargo, la mayor parte del desarrollo web se realiza en un lenguaje de programación dinámico como Ruby o PHP utilizando un editor de código fuente o un IDE. El HTML generado por todos los sitios web, excepto los estáticos más simples, se almacena como archivos de plantilla individuales que se ensamblan mediante el software que controla el sitio y no componen un documento HTML completo por sí mismos.

TeX

Los matemáticos, físicos y científicos de la computación a menudo producen artículos y libros utilizando TeX o LaTeX en archivos de texto plano. Dichos documentos suelen crearse con un editor de texto estándar, aunque algunas personas utilizan editores especializados para TeX.

Editor colaborativo (*Collaborative editor*)

Permite que múltiples usuarios trabajen en el mismo documento de forma simultánea desde ubicaciones remotas a través de una red. Los cambios realizados por cada usuario individual se rastrean y se fusionan automáticamente en el documento para eliminar la posibilidad de ediciones en conflicto. Estos editores también suelen incluir un componente de chat en línea para la discusión entre los editores.

Editor libre de distracciones (*Distraction-free editor*)

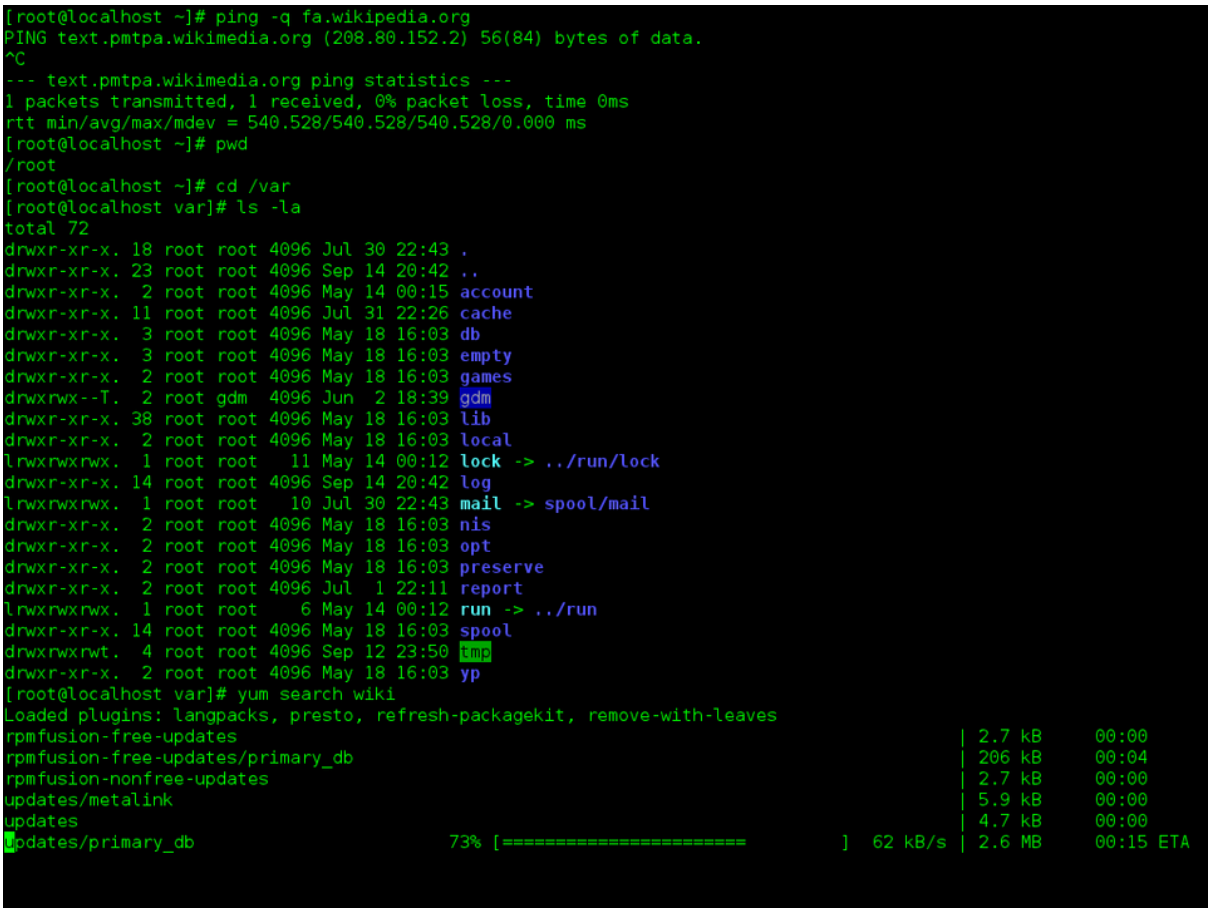
Proporciona una interfaz minimalista con el propósito de aislar al escritor del resto de las aplicaciones y del sistema operativo, permitiéndole así concentrarse en la escritura sin distracciones provenientes de elementos de la interfaz como la barra de herramientas o el área de notificaciones.

Editor programable (*Programmable editor*)

Se puede mejorar o extender para realizar múltiples funciones, mientras que los editores más simples se enfocan en una sola o, como en el caso de gPHPedit, están orientados a un único lenguaje de programación.

Interfaz de línea de comandos (Command-line interface)(CLI)

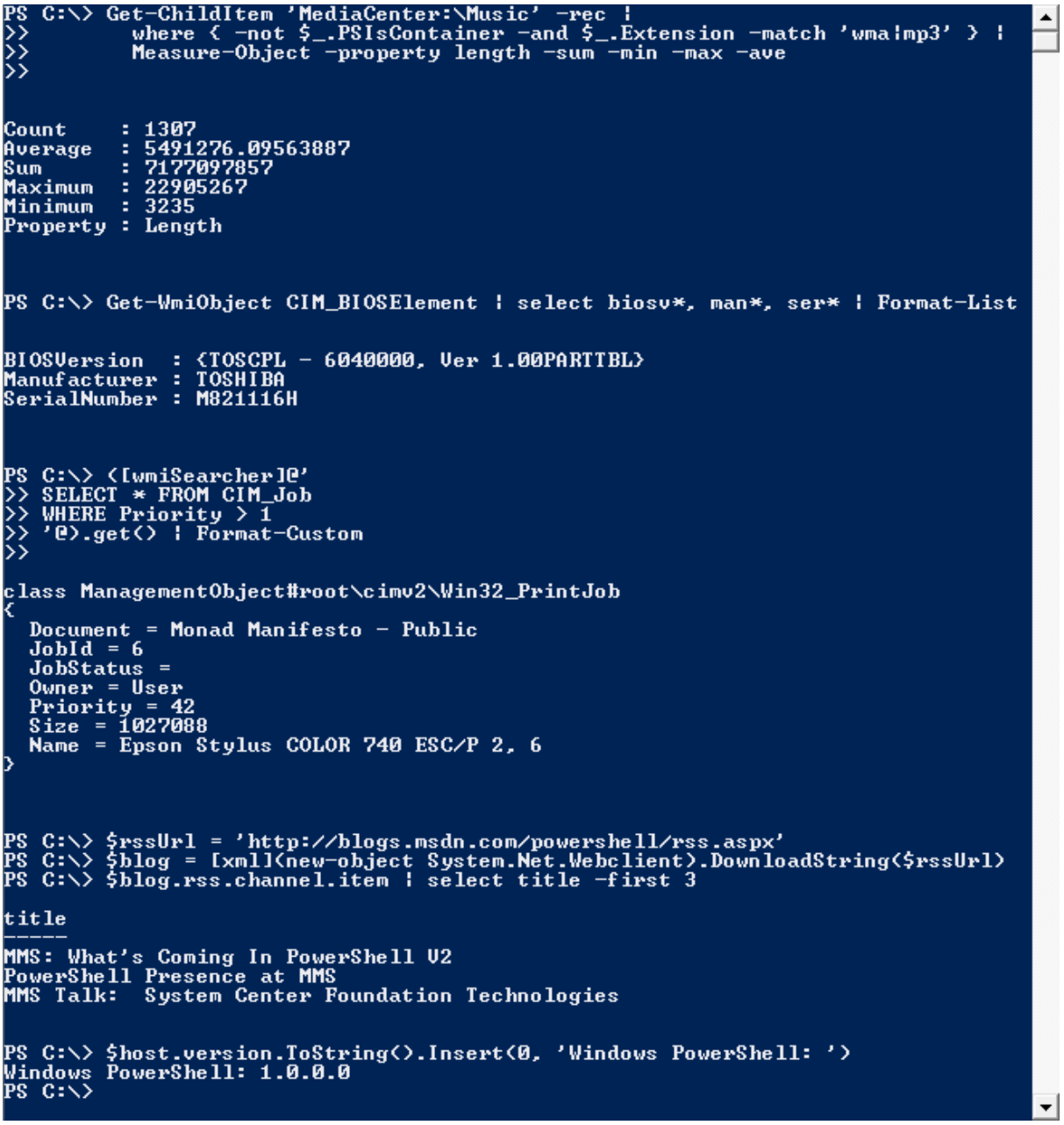
Una **interfaz de línea de comandos** (CLI, por sus siglas en inglés *Command-Line Interface*), también conocida como *shell* de línea de comandos, es un medio para interactuar con un programa informático mediante comandos, cada uno formateado como una línea de texto.



Captura de pantalla de una sesión de muestra de Bash en GNOME Terminal 3, Fedora 15.

El concepto de interactuar con una computadora a través de texto evolucionó a lo largo de dos décadas, pasando del hardware físico a software especializado. Antes de la CLI, las computadoras se programaban mediante interruptores físicos o tarjetas perforadas. El cambio hacia una interacción basada en un estilo de "comandos" comenzó con el uso de teleimpresores (*teletypes*). Sistemas pioneros como el Whirlwind I (1951) en el MIT

comenzaron a utilizar entradas similares a las de una máquina de escribir para control directo, alejándose del procesamiento por lotes (*batch processing*), donde se entregaba un paquete de tarjetas a un operador y se esperaba horas por un resultado. Dispositivos como el Teletype Modelo 33, introducido en 1963, permitían a los operadores escribir un comando y recibir una respuesta impresa directamente desde la computadora.



Captura de pantalla de Windows PowerShell 1.0, ejecutándose en Windows Vista.

El interprete de línea de comandos surgió cuando las computadoras se volvieron lo suficientemente potentes como para gestionar a múltiples usuarios a la vez, lo que se conoce como tiempo compartido (*time-sharing*). El Sistema de Tiempo Compartido Compatible (CTSS, por sus siglas en inglés *Compatible Time-Sharing System*), desarrollado en 1961 en el MIT, introdujo el primer interprete de línea de comandos real. Por primera vez, un usuario podía escribir un comando como `LOGIN` o `PRINT`, y un programa dedicado (el supervisor) analizaba ese texto y ejecutaba la función correspondiente. En 1964, Louis

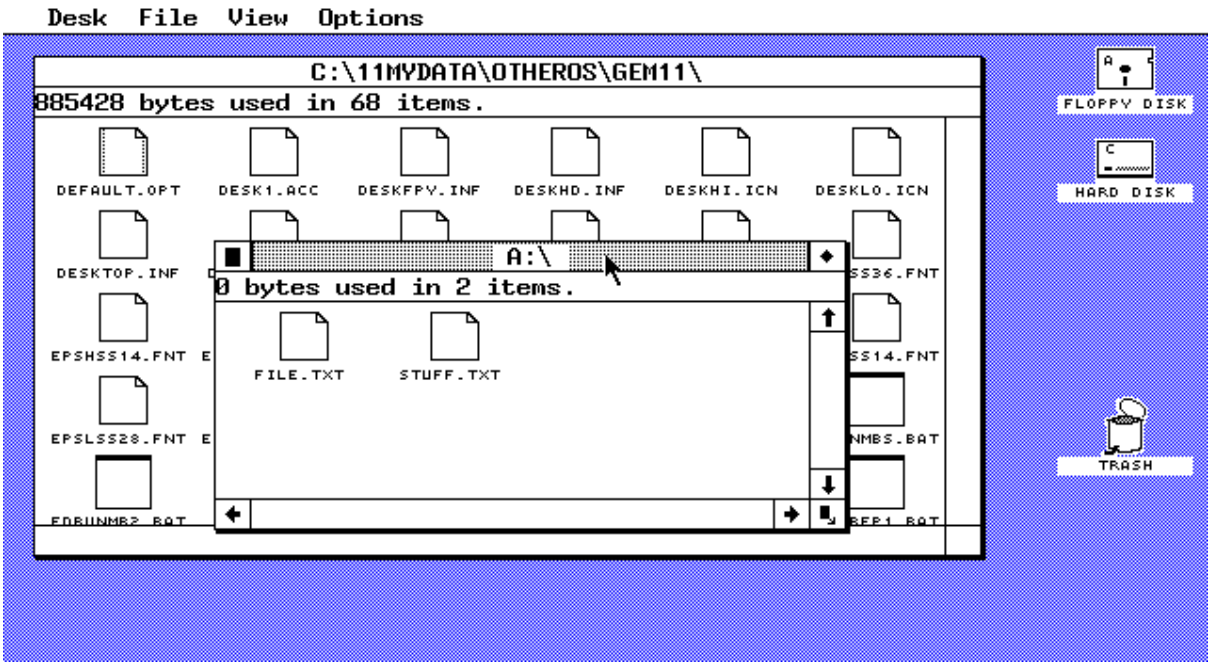
Pouzin introdujo el software que hoy reconocemos como un "*shell*" para el sistema operativo Multics, proporcionando un entorno programable para la ejecución de scripts. Muchas utilidades para la gestión de sistemas operativos y el desarrollo de software proporcionan interfaces de línea de comandos (CLIs).

Muchas CLIs permiten la automatización de software, ya que los comandos se pueden almacenar en un archivo de script que se puede utilizar repetidamente. Un script permite que los comandos contenidos en él se ejecuten como un grupo (como un único programa).

Las alternativas a una CLI incluyen una interfaz gráfica de usuario o GUI (incluyendo la metáfora de escritorio como Windows), menús basados en texto (incluyendo DOS Shell y SMIT de IBM AIX) y atajos de teclado.

Comparación con las interfaces gráficas de usuario

En comparación con una interfaz gráfica de usuario, una interfaz de línea de comandos requiere menos recursos del sistema para implementarse. Dado que las opciones de los comandos se proporcionan mediante unos pocos caracteres en cada línea de comandos, a un usuario experimentado a menudo le resulta más fácil acceder a dichas opciones. La automatización de tareas repetitivas se simplifica mediante la edición de líneas y los mecanismos de historial para almacenar secuencias usadas reciente o frecuentemente, lo que permite revisar y repetir comandos; esto puede extenderse a un lenguaje de *scripting* capaz de recibir parámetros y opciones variables.



Captura de pantalla de una interfaz gráfica de usuario con íconos y ventanas (GEM 1.1 Desktop).

Un sistema de línea de comandos puede requerir manuales en línea o impresos para la consulta del usuario, aunque a menudo una opción de ayuda (*help*) proporciona una revisión concisa de las opciones de un comando. Es posible que el entorno de línea de comandos no ofrezca mejoras gráficas como diferentes fuentes tipográficas o ventanas de edición extendidas presentes en una GUI. Para un usuario nuevo, puede resultar difícil familiarizarse con todos los comandos y opciones disponibles, en comparación con los íconos y menús desplegables de una interfaz gráfica de usuario, sin recurrir a los manuales.

Tipos

Interfaces de línea de comandos de sistemas operativos

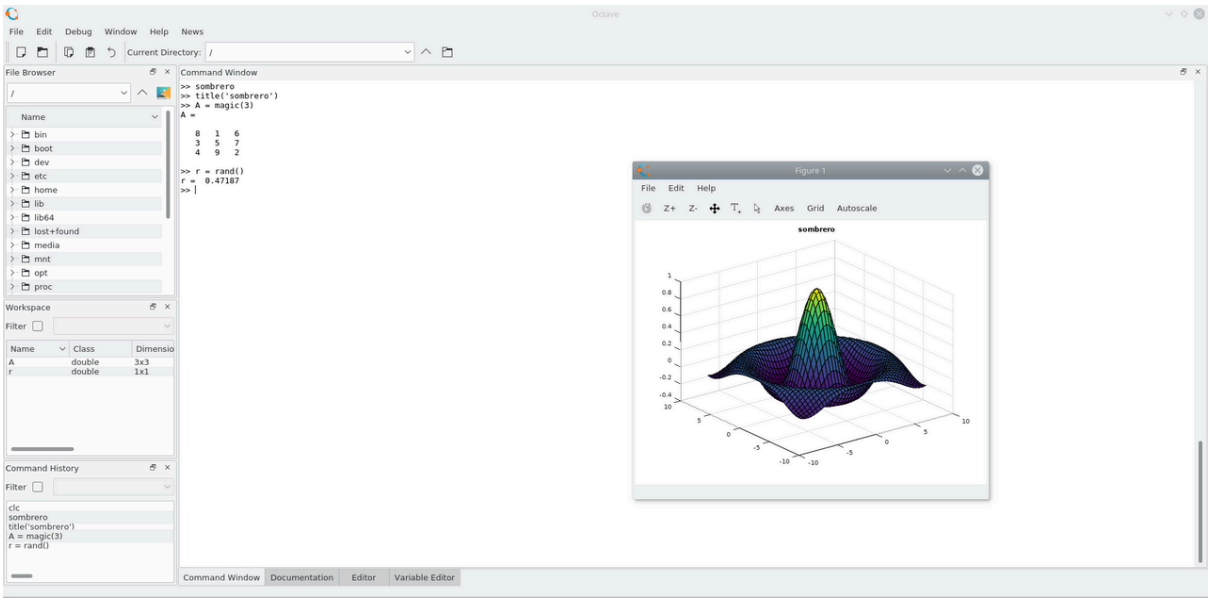
Las interfaces de línea de comandos de los sistemas operativos (SO) suelen ser programas independientes suministrados con el propio sistema operativo. Un programa que implementa dicha interfaz de texto se denomina frecuentemente intérprete de comandos, procesador de comandos o *shell*.

Ejemplos de intérpretes de línea de comandos son Nushell, DIGITAL Command Language (DCL) de DEC en OpenVMS y RSX-11, los diversos *shells* de Unix (`sh`, `ksh`, `csh`, `tcsh`, `zsh`, Bash, etc.), el CCP de CP/M, el `COMMAND.COM` de DOS, así como los programas `cmd.exe` de OS/2 y Windows; estos últimos grupos basados en gran medida en las CLI de RSX-11 y RSTS de DEC. En la mayoría de los sistemas operativos es posible reemplazar el programa *shell* predeterminado por alternativas; ejemplos de ello incluyen 4DOS para DOS, 4OS2 para OS/2 y 4NT / Take Command para Windows.

Aunque el término *shell* se utiliza a menudo para describir un intérprete de línea de comandos, en sentido estricto, un *shell* es un intérprete y procesador de comandos y, por extensión, un programa que proporciona una interfaz (ya sea una interfaz de usuario —como una interfaz de línea de comandos o GUI— o una interfaz de programación) para dicho intérprete y para las capacidades generales del sistema operativo. Por ejemplo, la GUI predeterminada de Windows es un *shell* gráfico denominado `EXPLORER.EXE`, según se define en la línea `SHELL=EXPLORER.EXE` del archivo de configuración `WIN.INI`. Este programa, estrictamente hablando, es un *shell* gráfico y, por extensión, un *shell* (en rigor, el *shell* de Windows es `shell32.dll`, no `explorer.exe`, que es la GUI para `shell32.dll`), pero no es un intérprete de línea de comandos (CLI) ni un *shell* de línea de comandos.

Interfaces de línea de comandos en aplicaciones

Los programas de aplicación (a diferencia de los sistemas operativos) también pueden contar con interfaces de línea de comandos.



Interfaz gráfica de usuario de GNU Octave con interfaz de línea de comandos.

Un programa de aplicación puede admitir ninguno, alguno o la totalidad de estos tres mecanismos principales de interfaz de línea de comandos:

- **Parámetros:** La mayoría de las interfaces de línea de comandos admiten un medio para pasar información adicional a un programa al momento de su ejecución.
- **Sesiones interactivas de línea de comandos:** Tras su inicio, un programa puede proporcionar al operador un medio independiente para ingresar comandos.
- **Comunicación entre procesos (*Inter-process communication*):** La mayoría de los sistemas operativos admiten medios de comunicación entre procesos (por ejemplo, flujos estándar o tuberías con nombre). Las líneas de comandos de los procesos cliente se pueden redirigir a un programa CLI mediante alguno de estos métodos.

Algunas aplicaciones admiten una CLI, presentando su propio indicador de comandos (*prompt*) al usuario y aceptando líneas de comandos. Otros programas admiten tanto una CLI como una GUI. En algunos casos, una GUI es simplemente un envoltorio (*wrapper*) alrededor de un archivo ejecutable CLI independiente. En otros casos, un programa puede proporcionar una CLI como una alternativa opcional a su GUI. Las CLI y las GUI a menudo admiten funcionalidades distintas. Por ejemplo, todas las características de MATLAB, un programa informático de análisis numérico, están disponibles a través de la CLI, mientras que la GUI de MATLAB solo expone un subconjunto de ellas.

En *Colossal Cave Adventure* (1975), el usuario utilizaba una CLI para ingresar una o dos palabras y así explorar un sistema de cuevas.

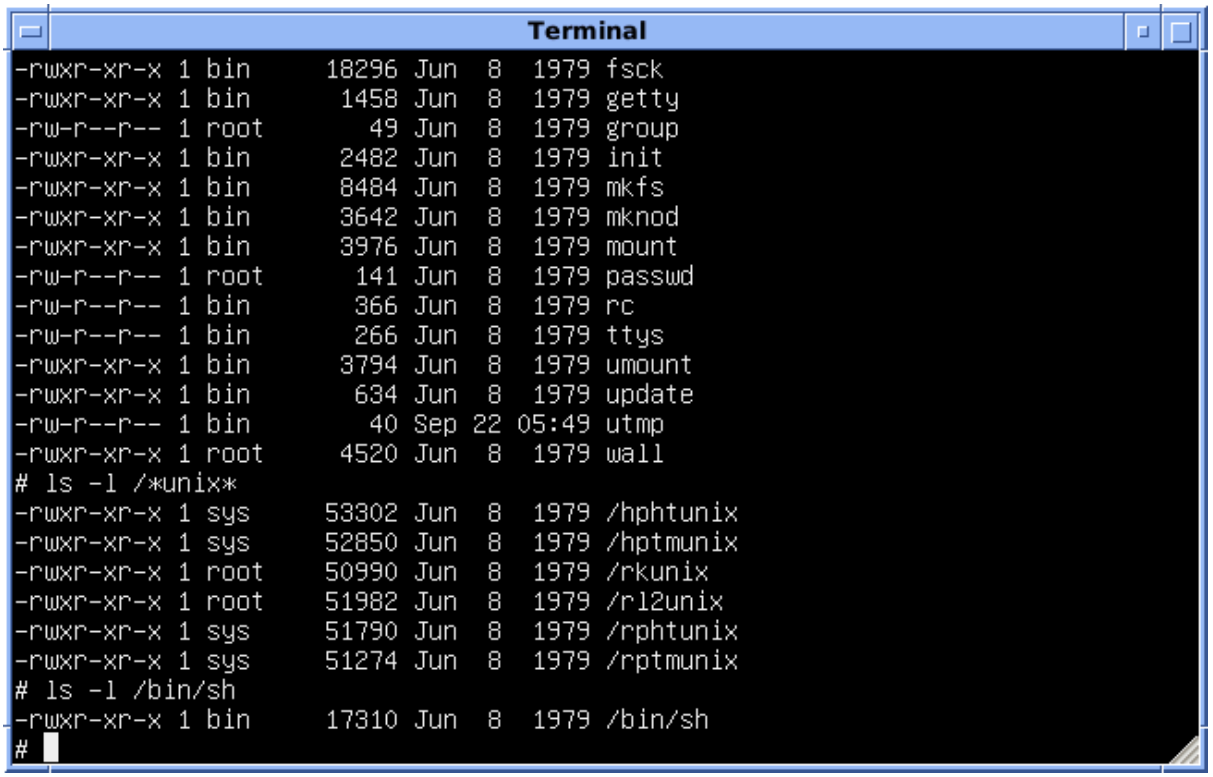
Historia

La interfaz de línea de comandos evolucionó a partir de una forma de comunicación llevada a cabo por personas mediante máquinas de teleimpresor o teletipo (TTY). A veces, estas interacciones implicaban el envío de una orden o una confirmación a través de telex. Los primeros sistemas informáticos solían utilizar un teleimpresor para interactuar con el operador.

El teleimpresor mecánico fue sustituido por un "teletipo de cristal" (*glass tty*), compuesto por un teclado y una pantalla que emulaban al teleimpresor. Las terminales "inteligentes" permitieron funciones adicionales, como el movimiento del cursor por toda la pantalla o la edición local de datos en la propia terminal antes de su transmisión a la computadora. A medida que la revolución de las microcomputadoras reemplazó la arquitectura tradicional de tiempo compartido (minicomputadora + terminales), las terminales físicas fueron reemplazadas por emuladores de terminal: software para PC que interpretaba las señales de la terminal enviadas a través de los puertos serie del equipo. Estos se utilizaban típicamente para conectar las nuevas computadoras personales de una organización con sus minicomputadoras o *mainframes* existentes, o para conectar PC con PC. Algunas de estas PC ejecutaban software de sistemas de tablón de anuncios (BBS).

Las primeras CLI de los sistemas operativos se implementaron como parte de los programas monitores residentes y no se podían reemplazar fácilmente. La primera implementación del *shell* como un componente reemplazable formó parte del sistema operativo de tiempo compartido Multics. En 1964, Louis Pouzin, miembro del personal del Centro de Cómputo del MIT, desarrolló la herramienta RUNCOM para ejecutar scripts de comandos permitiendo la sustitución de argumentos. Pouzin acuñó el término *shell* para describir la técnica de utilizar comandos a la manera de un lenguaje de programación, y escribió un artículo sobre cómo implementar esta idea en el sistema operativo Multics. Pouzin regresó a su Francia natal en 1965, y el primer *shell* para Multics fue desarrollado por Glenda Schroeder.

El primer *shell* de Unix, el V6 shell, fue desarrollado por Ken Thompson en 1971 en Bell Labs y se basó en el *shell* de Multics diseñado por Schroeder. El Bourne shell se introdujo en 1977 como un reemplazo para el V6 shell. Aunque se utiliza como un intérprete de comandos interactivo, también se concibió como un lenguaje de *scripting* y contiene la mayoría de las características necesarias para producir programas estructurados. El Bourne shell condujo al desarrollo del KornShell (ksh), el Almquist shell (ash) y el popular Bourne-again shell (o Bash).



Interacción del *Bourne shell* en Unix Versión 7.

Las primeras microcomputadoras se basaban en una interfaz de línea de comandos como CP/M, DOS o Applesoft BASIC. Durante las décadas de 1980 y 1990, con la llegada del Apple Macintosh y de Microsoft Windows en las PC, la interfaz de línea de comandos fue reemplazada por la interfaz gráfica de usuario como la interfaz principal. La línea de comandos permaneció disponible como una interfaz alternativa, utilizada a menudo por administradores de sistemas y usuarios avanzados para la administración del sistema, la programación y el procesamiento por lotes.

En noviembre de 2006, Microsoft lanzó la versión 1.0 de Windows PowerShell (anteriormente conocida bajo el código Monad), la cual combinó características de los *shells* tradicionales de Unix con su marco de trabajo orientado a objetos .NET Framework. MinGW y Cygwin son paquetes de código abierto para Windows que ofrecen una CLI de tipo Unix. Microsoft también ofreció la implementación de ksh de MKS Inc., el MKS Korn shell para Windows, a través de su complemento Services for UNIX.

Desde 2001, el sistema operativo Macintosh, macOS, se ha basado en un sistema operativo de tipo Unix llamado Darwin. En estas computadoras, los usuarios pueden acceder a una interfaz de línea de comandos de tipo Unix ejecutando el programa emulador de terminal llamado Terminal, el cual se encuentra en la subcarpeta Utilidades dentro de la carpeta Aplicaciones, o iniciando sesión de forma remota en el equipo mediante `ssh`. Z shell (zsh) es el *shell* predeterminado en macOS; también se proporcionan Bash, tcsh y KornShell. Antes de macOS Catalina, Bash era el *shell* predeterminado.

Uso

Una CLI se utiliza siempre que un vocabulario amplio de comandos o consultas, combinado con una gama variada (o arbitraria) de opciones, pueda ingresarse más rápidamente en forma de texto que con una interfaz gráfica pura (GUI). Este es típicamente el caso de los *shells* de comandos en sistemas operativos. Las CLI también se emplean en sistemas que carecen de recursos suficientes para soportar una interfaz gráfica de usuario. Algunos lenguajes de programación (como Python, Forth, LISP, Rexx y muchos dialectos de BASIC) proporcionan un modo de línea de comandos interactivo (generalmente un bucle de lectura-evaluación-impresión o REPL) para permitir una rápida evaluación de código.

Las CLI son utilizadas con frecuencia por programadores y administradores de sistemas, en entornos científicos e ingenieriles, y por usuarios avanzados de computadoras personales. Asimismo, las CLI son populares entre personas con discapacidades visuales, ya que los comandos y las respuestas se pueden mostrar mediante pantallas braille reconfigurables (*refreshable braille displays*).

Anatomía de una CLI de *shell*

El patrón general de una línea de comandos es:`prompt comando param1 param2 param3 … paramN`

- **`prompt` (indicador de comandos):** generado por el programa para indicar que está listo para recibir entradas; un *prompt* puede incluir información contextual y a menudo termina con un símbolo que proporciona una señal visual de que se espera una entrada del usuario.
- **`comando`:** introducido por el usuario; un comando suele pertenecer a una de estas dos clases:
    1. **Comandos internos:** son reconocidos y procesados directamente por el intérprete de línea de comandos. Los comandos internos también se denominan comandos integrados (*built-in commands*).
    2. **Comandos externos:** ejecutan archivos ejecutables independientes. El intérprete de línea de comandos busca archivos ejecutables cuyos nombres coincidan con el comando externo.
- **`param1 … paramN`:** parámetros proporcionados por el usuario. El formato y el significado de los parámetros dependen del comando ejecutado. En el caso de los comandos externos, los valores de los parámetros se entregan al programa en el momento en que el sistema operativo lo inicia. Los parámetros pueden ser tanto argumentos como opciones.

En este formato, los delimitadores entre los elementos de la línea de comandos son caracteres de espacio en blanco y el delimitador de fin de línea es el salto de línea. Esta es una convención ampliamente utilizada (aunque no universal).

En general, se puede considerar que una interfaz de línea de comandos consta de sintaxis y semántica. La **sintaxis** es la gramática que deben seguir todos los comandos. En el caso de los sistemas operativos, DOS y Unix definen cada uno su propio conjunto de reglas a seguir. En el caso de los sistemas embebidos, cada fabricante —como Nortel, Juniper Networks o Cisco Systems— define su propio conjunto de reglas propietarias. Estas reglas también dictan cómo navega el usuario a través del sistema de comandos. La **semántica** define qué tipo de operaciones son posibles, sobre qué tipo de datos se pueden realizar dichas operaciones y cómo representa la gramática estas operaciones y datos: el significado simbólico dentro de la sintaxis.

Dos CLI diferentes pueden coincidir en la sintaxis o en la semántica, pero solo cuando coinciden en ambas se pueden considerar lo suficientemente similares como para permitir que los usuarios utilicen ambas CLI sin necesidad de aprender nada nuevo, así como para posibilitar la reutilización de *scripts*.

Una CLI simple mostrará un indicador de comandos (*prompt*), aceptará una línea de comandos escrita por el usuario y finalizada con la tecla Intro (*Enter*), para luego ejecutar el comando especificado y ofrecer una visualización textual de los resultados o mensajes de error. Las CLI avanzadas validarán, interpretarán y expandirán los parámetros de la línea de comandos antes de ejecutar el comando especificado, y opcionalmente capturarán o redirigirán su salida.

A diferencia de un botón o elemento de menú en una GUI, una línea de comandos es típicamente autodocumentada, ya que declara exactamente lo que el usuario desea realizar. Además, las líneas de comandos suelen incluir múltiples valores predeterminados que pueden modificarse para personalizar los resultados. Las líneas de comandos útiles se pueden guardar asignando una cadena de caracteres o un alias para representar el comando completo, o bien se pueden agrupar varios comandos para realizar una secuencia más compleja —por ejemplo, compilar el programa, instalarlo y ejecutarlo— creando una entidad única llamada procedimiento de comandos o *script*, el cual puede tratarse a su vez como un comando. Estas ventajas implican que el usuario debe descifrar un comando o serie de comandos complejos una sola vez, ya que puede guardarlos para utilizarlos de nuevo.

Los comandos proporcionados a un *shell* de CLI suelen adoptar una de las siguientes formas:

- `hacerAlgo como aArchivos`
- `hacerAlgo como archivoOrigen archivoDestino`
- `hacerAlgo como < archivoEntrada > archivoSalida`
- `hacerAlgo como | hacerAlgo como | hacerAlgo como > archivoSalida`

donde `hacerAlgo` es, en efecto, un verbo; `como` actúa como un adverbio (por ejemplo, si el comando debe ejecutarse de forma detallada/*verbose* o silenciosa/*quiet*); y `aArchivos` representa el objeto u objetos (típicamente uno o más archivos) sobre los cuales debe actuar el comando.

El carácter `>` en el tercer ejemplo es un operador de redirección que indica al intérprete de línea de comandos que envíe la salida del comando no a su salida estándar habitual (la pantalla), sino al archivo especificado; esto sobrescribirá el archivo. El uso de `>>` redirigirá la salida agregándola al final del archivo existente. Otro operador de redirección es la barra vertical (`|`), que crea una tubería (*pipeline*) en la que la salida de un comando se convierte en la entrada del siguiente comando.

CLI y protección de recursos

En algunos sistemas, como los de tipo Unix o Windows, es posible modificar el conjunto de comandos disponibles alterando las rutas incluidas en la variable de entorno `PATH` o su equivalente. En los sistemas de tipo Unix, el archivo que contiene el ejecutable o el *script* para un comando también debe contar con permisos de ejecución. Los directorios especificados en la variable de entorno se buscan en el orden exacto en el que han sido declarados. Al reordenar la ruta, un usuario puede ejecutar, por ejemplo, `\OS2\MDOS\E.EXE` en lugar de `\OS2\E.EXE`, cuando la configuración predeterminada indica lo contrario. Cambiar el nombre de los ejecutables también es una práctica efectiva: por ejemplo, los usuarios suelen renombrar su editor preferido como `EDIT`.

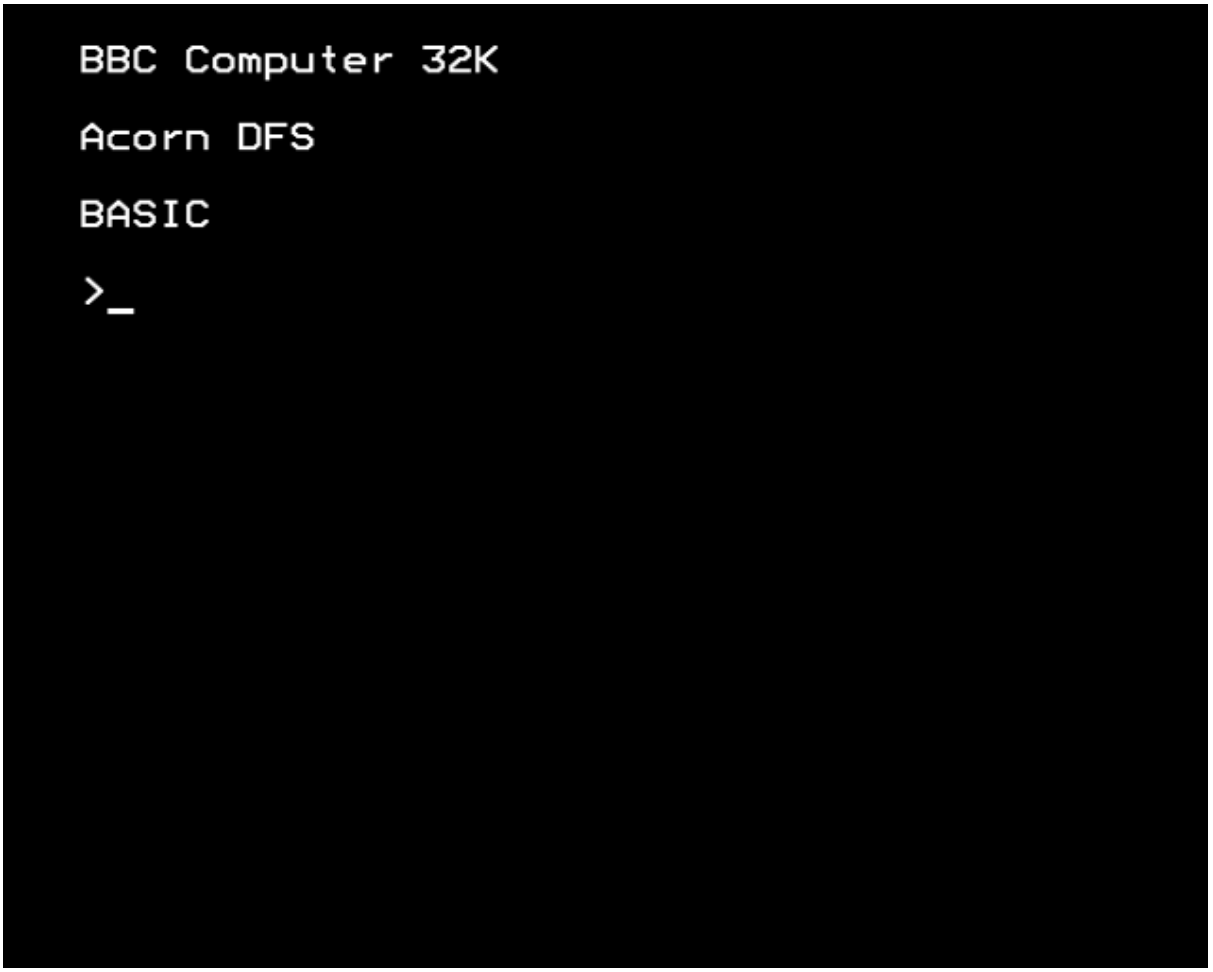
La línea de comandos permite restringir los comandos disponibles, como limitar el acceso a comandos internos avanzados. El Bourne shell y sus derivados (como Bash) se pueden ejecutar en modo restringido (*restricted shell*); entre otras cosas, esto prohíbe al usuario modificar la variable de entorno `PATH`, de modo que si `PATH` se configura para incluir únicamente directorios con comandos permitidos, el usuario solo podrá ejecutar dichos comandos. El `cmd.exe` de Windows también admite la restricción de comandos disponibles. A menudo, los programas de tipo *shareware* limitan la variedad de comandos disponibles, mostrando mensajes en el *prompt* como «su administrador ha deshabilitado la ejecución de archivos por lotes».

Algunas CLI, como las presentes en los enrutadores de red, cuentan con una jerarquía de modos donde cada uno admite un conjunto diferente de comandos. Los comandos se agrupan según su asociación con la seguridad, el sistema, las interfaces, entre otros aspectos. En estos sistemas, el usuario puede desplazarse a través de una serie de submodos. Por ejemplo, si la CLI dispone de dos modos llamados *interface* y *system*, el usuario podría emplear el comando `interface` para ingresar al modo de interfaz; en este punto, los comandos del modo *system* no estarán accesibles hasta que el usuario salga del modo de interfaz e ingrese al modo de sistema.

En los sistemas operativos insignias MVS de IBM, la **Authorised Program Facility (APF)** permite que ciertos programas autorizados se ejecuten con privilegios elevados, algo análogo al UID(0) en los sistemas de tipo Unix. TSO/E admite la llamada a comandos autorizados, pero únicamente para aquellos comandos, programas y servicios incluidos en una lista controlada por la instalación.

Indicador de comandos (*Command prompt*)

Un indicador de comandos (o simplemente *prompt*) es una secuencia de uno o más caracteres utilizada en una interfaz de línea de comandos para señalar la disponibilidad del sistema para recibir órdenes. Literalmente, solicita (*prompts*) al usuario que realice una acción. Un *prompt* suele finalizar con uno de los caracteres `$`, `%`, `#`, `:`, `>` o `-`, y a menudo incluye información adicional, como la ruta del directorio de trabajo actual y el nombre del equipo (*hostname*).



Prompt de una BBC Micro después del encendido o de un reinicio forzado (*hard reset*).

En muchos sistemas Unix y sus derivados, el *prompt* finaliza comúnmente en `$` o `%` si el usuario es un usuario estándar, pero cambia a `#` si se trata de un superusuario (*root* en la terminología de Unix).

Los usuarios finales suelen tener la posibilidad de modificar los *prompts*. Dependiendo del entorno, pueden incluir colores, caracteres especiales y otros elementos (como variables y funciones para la hora actual, el usuario, el número de *shell* o el directorio de trabajo) para hacer que el indicador sea más informativo o visualmente atractivo, diferenciar sesiones en

múltiples máquinas o indicar el nivel actual de anidamiento de los comandos. En algunos sistemas, se pueden emplear tokens especiales en la definición del *prompt* para lograr que el intérprete de línea de comandos invoque programas externos al momento de mostrarlo.

En el `COMMAND.COM` de DOS y en el `cmd.exe` de Windows NT, los usuarios pueden modificar el *prompt* ejecutando el comando `PROMPT` o cambiando directamente el valor de la variable de entorno correspondiente `%PROMPT%`. La configuración predeterminada de la mayoría de los sistemas modernos, con el estilo `C:\>`, se obtiene, por ejemplo, mediante `PROMPT $P$G`. El estilo predeterminado de los sistemas DOS más antiguos, `C>`, se obtiene simplemente con `PROMPT`, aunque en algunos sistemas esto genera el estilo más reciente `C:\>`, a menos que se utilice en unidades de disquete `A:` o `B:`; en dichos sistemas se puede emplear `PROMPT $N$G` para anular el valor predeterminado automático y cambiar explícitamente al estilo antiguo.

Ejemplo de visualizaciones del sistema de *prompt*.



Toda la información que puede mostrar el sistema de *prompt* de comandos «Liquidprompt», aquí utilizando su tema «Jolly».



Ejemplo de visualización de Liquidprompt, utilizando su tema predeterminado.



Ejemplo de visualización de Liquidprompt, utilizando su tema «Pureship».



Ejemplo de visualización de Liquidprompt, utilizando su tema «Powerline».

Muchos sistemas Unix cuentan con la variable `$PS1` (*Prompt String 1*), aunque otras variables también pueden afectar al *prompt* (dependiendo del *shell* utilizado). En el *shell* Bash, un *prompt* con la forma:

```
[hora] usuario@equipo: dir_trabajo $
```

se podría configurar ejecutando el comando:

```
export PS1='[\t] \u@\H: \W $'
```

En `zsh`, la variable `$RPROMPT` controla un *prompt* opcional en el lado derecho de la pantalla. No es un *prompt* real en el sentido de que la ubicación de la entrada de texto no cambia; se utiliza para mostrar información en la misma línea que el *prompt*, pero alineada a la derecha.

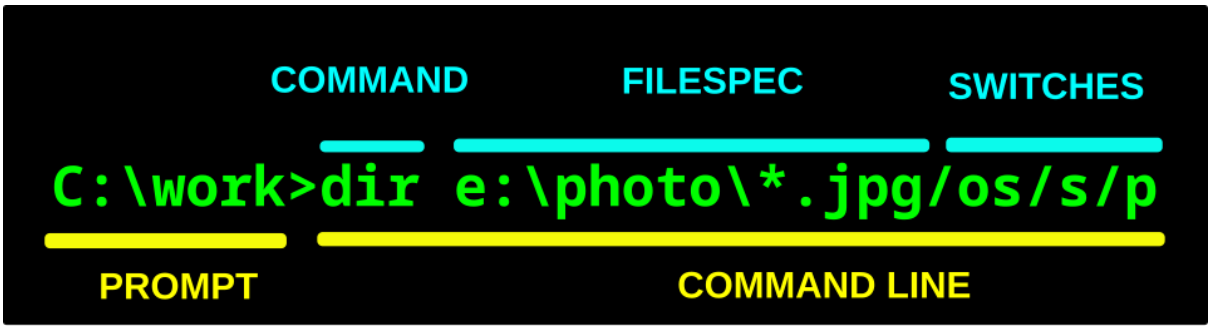
En RISC OS, el indicador de comandos es el símbolo `*`, por lo que los comandos de la CLI del sistema operativo a menudo se denominan *star commands* (comandos estrella). También se puede acceder a estos mismos comandos desde otras líneas de comandos (como la de BBC BASIC) anteponiendo un `*` al comando.

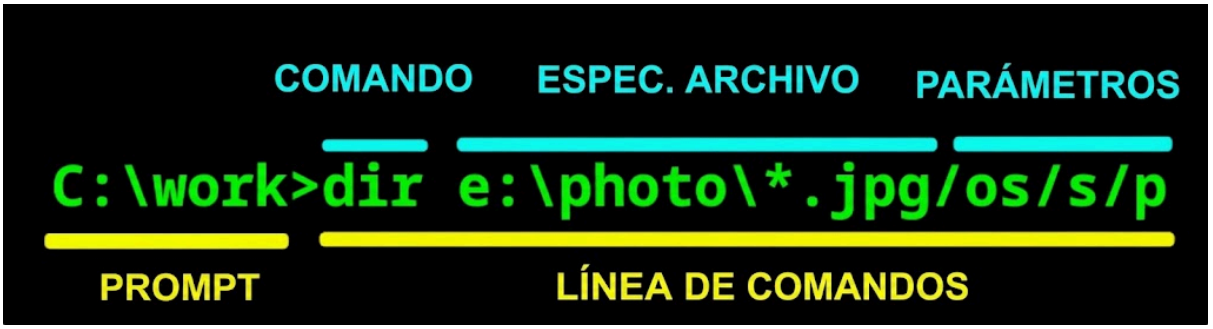
En el TSO de IBM, el indicador de comandos se denomina *READY prompt*, por la palabra `READY` que figura en el propio indicador.

Algunos *shells*, como Bash, `zsh` y PowerShell, permiten el uso de sistemas de *prompt* que muestran mucha más información en el indicador de comandos, como el tipo de conexión remota, el estado de los sensores del equipo o información adicional sobre un sistema de control de versiones. A menudo utilizan varios colores ANSI, caracteres Unicode y características avanzadas del *shell* para configurar una presentación vistosa y colorida.

Argumentos (*Arguments*)

Un argumento o parámetro de línea de comandos es un elemento de información que se le proporciona a un programa al momento de su inicio. Un programa puede recibir múltiples argumentos de línea de comandos que identifiquen fuentes o destinos de información, o que alteren el funcionamiento del propio programa.





Una línea de comandos de MS-DOS, que ilustra la descomposición (*parsing*) en comando y argumentos.

Cuando un procesador de comandos está activo, un programa se invoca típicamente escribiendo su nombre seguido de los argumentos de línea de comandos (si los hay). Por ejemplo, en entornos Unix y de tipo Unix, un ejemplo de argumento de línea de comandos es:

```
rm archivo.s
```

donde `archivo.s` es un argumento de línea de comandos que le indica al programa `rm` que elimine el archivo denominado `archivo.s`.

Algunos lenguajes de programación, como C, C++ y Java, permiten que un programa interprete los argumentos de la línea de comandos manejándolos como parámetros de cadena de texto en la función principal (`main`). Otros lenguajes, como Python, exponen la API específica del sistema operativo a través del módulo `sys`, y en particular mediante `sys.argv` para los argumentos de línea de comandos.

En los sistemas operativos de tipo Unix, un único guion `-` utilizado en lugar del nombre de un archivo es un valor especial que especifica que el programa debe procesar los datos provenientes de la entrada estándar (*standard input*) o enviar los datos a la salida estándar (*standard output*).

Opción de línea de comandos (*Command-line option*)

Una opción de línea de comandos o simplemente opción (también conocida como *flag* o *switch*) modifica la operación de un comando; el efecto lo determina el programa del comando. Las opciones siguen al nombre del comando en la línea de comandos, separadas, por ejemplo, por comas o espacios. Los separadores no siempre son obligatorios, como en `Dir/?` y `DIR /?` en DOS, los cuales tienen el mismo efecto de listar las opciones disponibles del comando `DIR`, mientras que `dir --help` (en muchas versiones de Unix) sí requiere que la opción esté precedida por al menos un espacio (y distingue entre mayúsculas y minúsculas).

El formato de las opciones varía ampliamente entre los sistemas operativos. En la mayoría de los casos, la sintaxis se rige por convención y no por un requisito del sistema operativo; la línea de comandos completa es simplemente una cadena de texto que se pasa a un programa, el cual puede procesarla de la manera que el programador desee, siempre que el intérprete pueda determinar dónde termina el nombre del comando y dónde comienzan sus argumentos y opciones.

Unos pocos ejemplos representativos de opciones de línea de comandos, la mayoría relacionados con listar archivos en un directorio, ilustran algunas convenciones:

Sistema operativo	Comando	Alternativa válida	Notas
OpenVMS	directory/owner	Dir /Owner	Instruye al comando *directory* para que también muestre el propietario de los archivos. *Nota:* El nombre del comando *Directory* no distingue entre mayúsculas y minúsculas, y se puede abreviar con tantas letras como sean necesarias para mantenerse único.

Windows	`DIR /Q /O:S d*`	`dir /q d* /o:s`	Muestra la propiedad de los archivos cuyos nombres comiencen con "d" (o "D"), ordenados por tamaño, del más pequeño al más grande. *Nota:* Los espacios alrededor del argumento `d*` son obligatorios.

Sistemas tipo Unix	`ls -lS D*`	`ls -S -l D*`	Muestra en formato largo los archivos y directorios cuyos nombres comiencen con "D" (pero no con "d"), ordenados por tamaño (del más grande al más pequeño). *Nota:* Se requieren espacios alrededor de todos los argumentos y opciones, pero algunas opciones se pueden agrupar; por ejemplo, `-lS` es equivalente a `-l -S`.

CLI de Data General RDOS	`list/e/s 04-26-80/b`	`List /S/E 4-26-80/B`	Lista cada atributo de los archivos creados antes del 26 de abril de 1980. *Nota:* El `/B` al final del argumento de fecha es un modificador local (*local switch*) que altera el significado de dicho argumento, mientras que `/S` y `/E` son modificadores globales (*global switches*), es decir, se aplican a todo el comando.

CLI de VM/CMS	`LISTFILE (FULL`	`DATE) l(ful`	Incluye en la lista la fecha en que el archivo se escribió por última vez. *Nota:* El nombre del comando `LISTFILE` no distingue entre mayúsculas y minúsculas, y se puede abreviar con tantas letras como sean necesarias para mantenerse único.
Comandos de operador de OS/360	`START TAPERDR,DSNAME=FOO.BAR`	`S TAPERDR,DSN=FOO.BAR`	Inicia el procedimiento o llamado `TAPERDR` con el nombre de conjunto de datos (*data set name*) proporcionado.

TSO	`LISTCAT LEVEL(FOO) MEMBERS`	`LISTC L(FOO) M`	Lista los conjuntos de datos en el nivel de índice especificado y enumera los miembros de cada PDS.
TSO	`SEND 'text' USER(FOO)`	`SE 'text' U(FOO)`	Envía un mensaje al usuario especificado.

El procesamiento o análisis sintáctico de argumentos de línea de comandos (*command-line argument parsing*) se utiliza para interpretar y estructurar los argumentos proporcionados a un programa, una funcionalidad que ofrecen diversos lenguajes de programación. Por ejemplo, `getopt()` forma parte de la biblioteca C POSIX para realizar el análisis de estos argumentos.

Abreviación de comandos

En Multics, las opciones de línea de comandos y las palabras clave de los subsistemas se pueden abreviar. Esta idea parece derivar del lenguaje de programación PL/I, con sus palabras clave acortadas (por ejemplo, `STRG` para `STRINGRANGE` y `DCL` para `DECLARE`). Por ejemplo, en el subsistema de foros de Multics, el parámetro `-long_subject` se puede abreviar como `-lgsj`. También es común que los comandos de Multics estén abreviados, lo cual corresponde típicamente a las letras iniciales de las palabras que se unen con guiones bajos para formar nombres de comandos, como el uso de `did` para `delete_iacl_dir`.

En otros sistemas, las abreviaturas son automáticas, como permitir la cantidad suficiente de los primeros caracteres del nombre de un comando para identificarlo de forma única (por ejemplo, `SU` como abreviatura de `SUPERUSER`), mientras que otros pueden tener abreviaturas específicas preprogramadas (por ejemplo, `MD` para `MKDIR` en `COMMAND.COM`) o definidas por el usuario a través de *scripts* por lotes y alias (por ejemplo, `alias md mkdir` en `tcsh`).

Convenciones de opciones en DOS, Windows y OS/2

En DOS, OS/2 y Windows, diferentes programas invocados desde su `COMMAND.COM` o `CMD.EXE` (o sus comandos internos) pueden utilizar sintaxis distintas dentro del mismo sistema operativo. Por ejemplo:

- Las opciones pueden indicarse mediante cualquiera de los caracteres de modificación (*switch*): `/`, `-`, o se puede permitir cualquiera de los dos.
- Pueden o no distinguir entre mayúsculas y minúsculas.
- A veces las opciones y sus argumentos se escriben juntos, a veces separados por espacios en blanco, y a veces por un carácter, típicamente `:` o `=`; por lo tanto: `Prog -fNombreArchivo`, `Prog -f NombreArchivo`, `Prog -f:NombreArchivo`, `Prog -f=NombreArchivo`.
- Algunos programas permiten combinar opciones de un solo carácter; otros no. El *switch* `-fA` puede significar lo mismo que `-f -A`, o puede ser incorrecto, o incluso puede ser un parámetro válido pero totalmente diferente.

En DOS, OS/2 y Windows, la barra diagonal (`/`) es la más prevalente, aunque el guion-menos (`-`) también se utiliza a veces. En muchas versiones de DOS (MS-DOS/PC DOS 2.xx y superiores, todas las versiones de DR-DOS desde la 5.0, así como PTS-DOS, Embedded DOS, FreeDOS y RxDOS), el carácter de modificación o *switch* (a veces abreviado como *switchar* o *switchchar*) que se debe utilizar viene definido por el valor devuelto por una llamada al sistema (`INT 21h/AX=3700h`). El carácter predeterminado devuelto por esta API es `/`, pero se puede cambiar a un guion-menos en los sistemas mencionados anteriormente, excepto en Datalight ROM-DOS y MS-DOS/PC DOS 5.0 y superiores, los cuales siempre devuelven `/` en esta llamada (a menos que se cargue uno de los muchos TSR disponibles para volver a habilitar la función *SwitChar*).

En algunos de estos sistemas (MS-DOS/PC DOS 2.xx, DOS Plus 2.1, DR-DOS 7.02 y superiores, PTS-DOS, Embedded DOS, FreeDOS y RxDOS), la configuración también se puede preconfigurar mediante una directiva `SWITCHAR` en el archivo `CONFIG.SYS`. El sistema Embedded DOS de General Software proporciona el comando `SWITCH` para este mismo propósito, mientras que 4DOS permite cambiar la configuración mediante `SETDOS /W:n`.

Bajo DR-DOS, si la configuración se ha cambiado de `/`, el primer separador de directorio `\` en la visualización del parámetro `PROMPT $G` cambiará a una barra diagonal `/` (que también es un separador de directorio válido en DOS, FlexOS, 4680 OS, 4690 OS, OS/2 y Windows), sirviendo así como una señal visual para indicar el cambio. Además, la configuración actual también se refleja en las pantallas de ayuda integradas. Algunas

versiones del `COMMAND.COM` de DR-DOS admiten el token `$/` en el `PROMPT` para mostrar la configuración actual. Asimismo, `COMMAND.COM` desde DR-DOS 7.02 proporciona una variable de pseudo-entorno llamada `%/%` para permitir la escritura de archivos por lotes (*batch jobs*) portables. Adicionalmente, varios comandos externos de DR-DOS admiten la variable de entorno `%SWITCHAR%` para anular la configuración del sistema.

Sin embargo, muchos programas están programados de forma rígida (*hardwired*) para utilizar únicamente `/`, en lugar de consultar la configuración del *switch* antes de analizar los argumentos de la línea de comandos. Un número muy reducido de programas, principalmente adaptaciones (*ports*) de sistemas de tipo Unix, están programados para aceptar `-` aunque el carácter de *switch* no esté configurado como tal (por ejemplo, `netstat` y `ping`, incluidos con Microsoft Windows, aceptan la opción `/?` para listar las opciones disponibles, y aun así la lista especificará la convención basada en `-`).

Convenciones de opciones en sistemas de tipo Unix

En los sistemas de tipo Unix, el guion-menos ASCII indica el inicio de las opciones; la convención más reciente (y de GNU) consiste en utilizar dos guiones seguidos de una palabra (por ejemplo, `--create`) para identificar la función de la opción, mientras que la convención antigua (que aún se mantiene como alternativa para opciones de uso frecuente) utiliza un solo guion seguido de una sola letra (por ejemplo, `-c`). Si a un solo guion le siguen dos o más letras, esto puede significar que se están especificando múltiples opciones de forma agrupada, o que la segunda letra y las subsiguientes constituyen un parámetro (como un nombre de archivo o una fecha) para la primera opción.

Dos caracteres de guion-menos sin letras a continuación (`--`) pueden indicar que los argumentos restantes no deben tratarse como opciones; esto resulta útil, por ejemplo, si el nombre de un archivo comienza con un guion o si los argumentos adicionales están destinados a un comando interno (por ejemplo, al utilizar `sudo`). Los guiones-menos dobles también se emplean a veces como prefijo para opciones largas con nombres más descriptivos, una característica habitual en el software de GNU. La función y programa `getopt`, así como el comando `getopts`, se utilizan habitualmente para procesar y analizar las opciones de la línea de comandos.

Los nombres de comandos, argumentos y opciones en Unix distinguen entre mayúsculas y minúsculas (salvo algunas excepciones puntuales, principalmente en casos donde se han adaptado a Unix comandos populares provenientes de otros sistemas operativos).

Convenciones de opciones en otros sistemas

- **FlexOS, 4680 OS y 4690 OS** utilizan el guion `-` para las opciones.
- **CP/M** utilizaba típicamente el corchete de apertura `[`.
- **Conversational Monitor System (CMS)** utiliza un único paréntesis izquierdo para separar las opciones al final del comando de los demás argumentos. Por ejemplo, en el siguiente comando, las opciones indican que el archivo de destino debe reemplazarse si ya existe, y que se debe conservar la fecha y hora del archivo de origen en la copia:`COPY archivo origen a archivo destino b (REPLACE OLDDATE`
- **Los comandos de operador de OS/360** utilizan una lista de parámetros posicionales y de palabras clave separados por comas; solo son válidas las abreviaturas documentadas de comandos y palabras clave. Los valores de las palabras clave se separan de estas mediante signos de igual (`=`).

Los comandos de TSO utilizan una lista de parámetros posicionales y de palabras clave separados por espacios; las palabras clave se pueden truncar a cualquier subcadena única. Solo son válidas las abreviaturas de comandos documentadas. Los valores de las palabras clave se encierran entre paréntesis.

La CLI de Data General en sus sistemas operativos como RDOS, AOS, etc., así como la versión de CLI incluida con su Business Basic, utiliza únicamente `/` como carácter de modificación (*switch*), no distingue entre mayúsculas y minúsculas y permite modificadores locales en algunos argumentos para controlar la forma en que se interpretan; por ejemplo, en `MAC/U LIB/S A B C $LPT/L`, la opción global `/U` indica al ensamblador de macros que agregue los símbolos del usuario, mientras que contiene dos modificadores locales: uno para especificar que `LIB` debe omitirse en la segunda pasada (`/S`) y otro para dirigir la impresión del listado a la impresora (`$LPT/L`).

Publicidad Donaciones

Ayuda de uso integrada (*Built-in usage help*)

Una de las críticas principales hacia las CLI es la falta de indicaciones visuales para el usuario sobre las acciones disponibles. Por el contrario, las GUI suelen informar al usuario sobre las opciones posibles mediante menús, íconos u otras señales visuales. Para superar esta limitación, muchos programas CLI muestran un mensaje de uso (*usage message*), típicamente cuando se invocan sin argumentos o con alguno de los parámetros `?`, `-?`, `-h`, `-H`, `/?`, `/h`, `/H`, `/Help`, `-help` o `--help`.

Sin embargo, ingresar el nombre de un programa sin parámetros con la esperanza de que muestre la ayuda de uso puede resultar riesgoso, ya que aquellos programas y *scripts* para los cuales los argumentos de línea de comandos son opcionales se ejecutarán inmediatamente sin previo aviso.

Aunque resulta deseable (al menos para el parámetro de ayuda), es posible que algunos programas no admitan todos los caracteres de inicio de opción ejemplificados anteriormente. En DOS, donde el carácter predeterminado para las opciones de línea de comandos puede cambiarse de `/` a `-`, los programas pueden consultar la API *SwitChar* para determinar la configuración actual. Por lo tanto, si un programa no está programado de forma rígida (*hardwired*) para admitirlos todos, el usuario podría necesitar conocer la configuración actual incluso para poder solicitar ayuda de manera confiable. Si el *SwitChar* se ha cambiado a `-` y, por ende, el carácter `/` se acepta como un separador de ruta alternativo en la línea de comandos de DOS, los programas podrían malinterpretar opciones como `/h` o `/H` como rutas en lugar de parámetros de ayuda. Sin embargo, si se proporciona como el primer o único parámetro, la mayoría de los programas de DOS, por convención, lo aceptarán como una solicitud de ayuda independientemente de la configuración actual de *SwitChar*.

En algunos casos, se pueden seleccionar diferentes niveles de ayuda para un programa. Algunos programas que admiten esto permiten proporcionar un nivel de detalle (*verbosity level*) como un argumento opcional para el parámetro de ayuda (como en `/H:1`, `/H:2`, etc.) o bien ofrecen una ayuda breve sobre los parámetros de ayuda con un signo de interrogación y una pantalla de ayuda más extensa para las demás opciones.

Dependiendo del programa, a veces es posible obtener ayuda adicional o más específica sobre los parámetros admitidos, ya sea proporcionando el parámetro en cuestión como un argumento para el parámetro de ayuda o viceversa (como en `/H:W` o en `/W:?`, asumiendo que `/W` sea otro parámetro admitido por el programa).

De forma similar al parámetro de ayuda, pero mucho menos común, algunos programas proporcionan información adicional sobre sí mismos (como el modo, estado, versión, autor, licencia o información de contacto) cuando se invocan con un parámetro de *acerca de* (*about*), como `-!`, `/!`, `-about` o `--about`.

Dado que los caracteres `?` y `!` típicamente también sirven para otros propósitos en la línea de comandos, es posible que no estén disponibles en todos los escenarios; por lo tanto, no deberían ser las únicas opciones para acceder a la información de ayuda correspondiente.



Salida del comando HELP de RT-11 en VT100.

El comando HELP en RT-11SJ (Single-Job Monitor) mostraba en pantalla una lista alfabética de comandos del sistema junto con sus respectivas descripciones resumidas. En terminales de tubo de rayos catódicos (CRT) como la clásica DEC VT100, la salida se organizaba en columnas (comando a la izquierda y función a la derecha), concluyendo al final con el *prompt* de comandos del sistema (típicamente el punto .).

Los comandos finales de la lista visible en esta interfaz de RT-11 incluyen:

Comando	**Descripción**
UNLOAD	Remueve un controlador de dispositivo (*device handler*) de la memoria.
UNSAV	Libera memoria de un programa guardado.

VDATE	Establece o muestra la fecha del sistema.
VTIME	Establece o muestra la hora del sistema.

Sintaxis de descripción de comandos (*Command description syntax*)

La ayuda de uso integrada y las páginas de manual (*man pages*) emplean comúnmente una sintaxis concisa para describir la forma válida de un comando:

- **Corchetes angulares (`< >`)** para parámetros obligatorios: `ping <nombre_de_host>`
- **Corchetes rectos (`[ ]`)** para parámetros opcionales: `mkdir [-p] <nombre_directorio>`
- **Puntos suspensivos (`...`)** para elementos que se repiten: `cp <origen1> [origen2...] <destino>`
- **Barras verticales (`|`)** para elegir entre opciones excluyentes: `netstat {-t|-u}`

Tenga en cuenta que estos caracteres tienen un significado diferente cuando se utilizan directamente en el *shell*. Los corchetes angulares pueden omitirse cuando no sea probable confundir el nombre del parámetro con una cadena literal.

El carácter de espacio (*The space character*)

En muchas áreas de la informática, pero particularmente en la línea de comandos, el carácter de espacio puede causar problemas ya que cumple dos funciones distintas e incompatibles: como parte de un comando o parámetro, o como separador de parámetros o nombres. La ambigüedad se puede evitar prohibiendo los espacios integrados en los nombres de archivos y directorios desde un principio (por ejemplo, sustituyéndolos por guiones bajos `_`), o encerrando entre comillas el nombre que contiene espacios o utilizando un carácter de escape antes del espacio, habitualmente una barra invertida (`\`). Por ejemplo:

```
Ruta larga/Nombre de programa largo Parámetro uno Parámetro dos …
```

es ambiguo (¿es "de programa largo" parte del nombre del programa, o son dos parámetros?); sin embargo:

```
Ruta_larga/Nombre_de_programa_largo Parámetro_uno Parámetro_dos …
```

```
RutaLarga/NombreDeProgramaLargo ParámetroUno ParámetroDos …
```

```
"Ruta larga/Nombre de programa largo" "Parámetro uno" "Parámetro dos" …
```

y

```
Ruta\ larga/Nombre\ de\ programa\ largo Parámetro\ uno Parámetro\ dos …
```

no son ambiguos. Los sistemas operativos de tipo Unix minimizan el uso de espacios integrados para reducir la necesidad de emplear comillas. En Microsoft Windows, a menudo es necesario usar comillas debido a que los espacios integrados (como en los nombres de directorios) son muy comunes.

Intérprete de línea de comandos (*Command-line interpreter*)

> *«Aunque la mayoría de los usuarios piensan en el shell como un intérprete de comandos interactivo, en realidad es un lenguaje de programación en el que cada sentencia ejecuta un comando. Dado que debe satisfacer tanto el aspecto interactivo como el de programación en la ejecución de comandos, es un lenguaje peculiar, moldeado tanto por la historia como por su diseño».*
>
> — **Brian W. Kernighan y Rob Pike**

El término **intérprete de línea de comandos** se aplica a los programas informáticos diseñados para interpretar una secuencia de líneas de texto, las cuales pueden ser ingresadas por un usuario o leídas desde un archivo u otro tipo de flujo de datos (*data stream*). El contexto de dicha interpretación suele ser el de un sistema operativo o un lenguaje de programación determinado.

Los intérpretes de línea de comandos permiten a los usuarios ejecutar diversas órdenes de una manera muy eficiente (y a menudo concisa). Esto requiere que el usuario conozca los nombres de los comandos, sus parámetros y la sintaxis del lenguaje que se está interpretando.

El mecanismo `#!` (*shebang*) de Unix y el comando `EXTPROC` de OS/2 facilitan el envío de archivos por lotes a procesadores externos. Se pueden emplear estos mecanismos para

escribir intérpretes de comandos específicos dedicados a usos particulares y procesar archivos de datos externos contenidos dentro de archivos por lotes.

Muchas interfaces gráficas, como el *Presentation Manager* de OS/2 y las primeras versiones de Microsoft Windows, utilizan líneas de comandos para invocar programas auxiliares (*helper programs*) con el fin de abrir documentos y aplicaciones. Los comandos se almacenan en el propio entorno gráfico o en archivos del sistema como el Registro de Windows o el archivo `OS2USER.INI` de OS/2.

Historia temprana (*Early history*)

Los ordenadores más antiguos no admitían dispositivos de entrada y salida interactivos, y a menudo dependían de conmutadores de estado (*sense switches*) y luces indicadoras para comunicarse con el operador de la computadora. Esto resultaba adecuado para los sistemas de procesamiento por lotes (*batch systems*) que ejecutaban un solo programa a la vez, donde con frecuencia el propio programador ejercía el rol de operador. Esta metodología ofrecía además la ventaja de un bajo consumo de recursos (*overhead*), ya que las luces y los conmutadores se podían probar y ajustar mediante una sola instrucción de máquina. Más adelante, se incorporó una consola del sistema unificada para permitir que el operador interactuara directamente con el sistema.



Un teleimpresor Teletype Model 33 ASR (*Automatic Send-Receive*) con teclado, lectora y perforadora de cinta de papel.

A partir de la década de 1960, la interacción del usuario con las computadoras se realizó principalmente a través de interfaces de línea de comandos; inicialmente en máquinas como el Teletype Model 33 ASR, y posteriormente en las primeras terminales de computadora basadas en tubos de rayos catódicos (CRT), como la VT52.



Terminal informático DEC VT52.

Todos estos dispositivos eran puramente textuales, sin capacidad para mostrar gráficos o imágenes. Para los programas de aplicaciones empresariales se utilizaban menús basados en texto, pero para la interacción general la interfaz principal era la línea de comandos.

Alrededor de 1964, Louis Pouzin introdujo el concepto y el nombre *shell* en Multics, basándose en herramientas previas y más simples del Compatible Time-Sharing System (CTSS).

A partir de principios de la década de 1970, el sistema operativo Unix adaptó el concepto de un entorno de línea de comandos potente e introdujo la capacidad de canalizar (*pipe*) la salida de un comando como entrada para otro. Unix también contaba con la capacidad de guardar y volver a ejecutar secuencias de comandos en forma de *scripts de shell*, los cuales funcionaban como comandos personalizados.

La línea de comandos también fue la interfaz principal para las primeras computadoras personales, tales como la Commodore PET, la Apple II y la BBC Micro, casi siempre en forma de un intérprete de BASIC. Cuando llegaron microcomputadoras más potentes orientadas al ámbito empresarial con CP/M y posteriormente computadoras DOS como la IBM PC, la línea de comandos comenzó a adoptar parte de la sintaxis y de las características de los *shells* de Unix, tales como la expansión de comodines (*globbing*) y la redirección de salidas mediante tuberías (*piping*).

La línea de comandos fue desafiada seriamente por primera vez por el enfoque de interfaz gráfica de usuario (GUI) de PARC, utilizado en la Apple Lisa de 1983 y en la Apple Macintosh de 1984. Algunos usuarios de computadoras utilizaron GUI como GEOS y Windows 3.1, pero la mayoría de los usuarios de IBM PC no reemplazaron su intérprete `COMMAND.COM` por una interfaz gráfica hasta el lanzamiento de Windows 95 en 1995.

Uso moderno como *shell* de sistema operativo

Aunque la mayoría de los usuarios no avanzados utilizan hoy en día una interfaz gráfica de usuario (GUI) de forma casi exclusiva, los usuarios con un nivel más avanzado disponen de potentes entornos de línea de comandos:

- **Sistemas de tipo Unix:** incluyen el *Bourne shell* y *shells* compatibles como Bash, además de otros *shells* propios de Unix.
    - **FreeBSD:** utiliza `tcsh` como su *shell* interactivo predeterminado para el superusuario (*root*), y `ash` como *shell* predeterminado para la ejecución de *scripts*.
    - **Distribuciones de Linux:** la gran mayoría incluye Bash de manera predeterminada.
    - **Apple macOS y algunas distribuciones de Linux:** emplean `zsh`. Anteriormente, macOS utilizaba `tcsh` y Bash.
    - **Sistemas Linux embebidos** (y otros sistemas de tipo Unix embebidos): suelen utilizar `ash`, integrado como parte de la suite Busybox.
    - **Android:** utiliza el *shell* `mksh`, el cual reemplazó a un *shell* derivado de `ash` que se empleaba en las versiones más antiguas de Android, complementado con comandos provenientes del binario independiente `toolbox`.
- **OpenVMS** utiliza el lenguaje de comandos de DIGITAL (*DIGITAL Command Language* o DCL).
    - Los *shells* de comandos de Unix se han adaptado (*ported*) a OpenVMS.
- **DOS y Windows** cuentan con varios entornos de línea de comandos:
    - `COMMAND.COM` es el intérprete de línea de comandos de MS-DOS, IBM PC DOS y Windows 9x, así como de clones como DR-DOS, SISNE plus, PTS-DOS, ROM-DOS y FreeDOS.
    - Windows NT incluye `cmd.exe`, y las versiones posteriores incorporan PowerShell.
    - El *Windows Resource Kit* y *Windows Services for UNIX* incluyen los *shells* Korn y Bourne junto con un intérprete de Perl (Services for UNIX contiene ActiveState ActivePerl en versiones posteriores, e Interix en las versiones 1 y 2 junto con un *shell* compilado por Microsoft).
    - Los usuarios de Windows pueden utilizar la interfaz CScript para ejecutar programas alternativos desde la línea de comandos. Las implementaciones del *shell* de Unix también están disponibles como parte del subsistema POSIX, Cygwin, MKS Toolkit, UWIN, Hamilton C shell y otros paquetes de

software. Los *shells* disponibles para estas herramientas de interoperabilidad incluyen `csh`, `ksh`, `sh`, Bash, `rsh`, `tclsh` y, con menos frecuencia, `zsh` y `psh`.
    - DCL se ha adaptado a sistemas Windows al menos en tres ocasiones, incluyendo PC-DCL y Acceler8 DCL Lite.
    - Existe otro `cmd.exe`, el cual es un *shell* simplificado para Windows CE 3.0.
    - Un intérprete de tipo MS-DOS llamado PocketDOS se adaptó para dispositivos con Windows CE; la versión más reciente es casi idéntica a MS-DOS 6.22 y también puede ejecutar Windows 1, 2 y 3.0, QBasic y otras herramientas de desarrollo, así como 4NT y 4DOS. La versión más reciente incluye varios *shells*, a saber: MS-DOS 6.22, PC DOS 7, DR DOS 3.xx, entre otros.
- **IBM OS/2** (y sus derivados como eComStation y ArcaOS) cuenta con el procesador `cmd.exe`. Este copia los comandos de `COMMAND.COM`, añadiendo extensiones para REXX.
- **Las implementaciones de PHP** incluyen un *shell* para uso interactivo denominado `php-cli`.
- **Tcl/Tk estándar** dispone de dos *shells* interactivos: `tclsh` y `wish`, siendo este último la versión gráfica (GUI).
- **Python, Ruby, Lua, XLNT** y otros intérpretes también poseen *shells* de comandos para su uso interactivo.
- **HarmonyOS, OpenHarmony y Oniro** emplean comandos provenientes del sistema de compatibilidad de caja de herramientas (*toolbox*) de terceros acoplado al núcleo Linux dentro del subsistema KAL y la biblioteca POSIX (`musl-libc`) con la ABI de Linux del núcleo HongMeng, junto con un *shell* predeterminado que incluye sus propios comandos ejecutables nativos.
- **Los enrutadores con Cisco IOS, Junos** y muchos otros se configuran comúnmente mediante la línea de comandos.
- **El sistema operativo Plan 9** utiliza el *shell* `rc`, cuyo diseño es similar al del *Bourne shell*.

Programación de scripts (*Scripting*)

La mayoría de los intérpretes de línea de comandos admiten la creación de *scripts* (*scripting*) en diversos grados. Al fin y al cabo, son intérpretes de un lenguaje de programación interpretado, aunque en muchos casos el lenguaje es exclusivo de ese intérprete de línea de comandos en particular. Estos entornos interpretan *scripts* —denominados indistintamente *shell scripts* o archivos por lotes (*batch files*)— escritos en el lenguaje específico que interpretan. Algunos intérpretes de línea de comandos también incorporan los motores de interpretación de otros lenguajes (como REXX), además del propio, lo que permite la ejecución directa de *scripts* escritos en esos otros lenguajes dentro del propio intérprete.

A la inversa, los lenguajes de programación para *scripting*, en particular aquellos que disponen de una función `eval` (como REXX, Perl, Python, Ruby o Jython), pueden emplearse para implementar intérpretes de línea de comandos y filtros. Para algunos sistemas operativos, muy especialmente en DOS, este tipo de intérprete de comandos proporciona una interfaz de línea de comandos mucho más flexible que la suministrada por el sistema. En otros casos, un intérprete de este tipo puede ofrecer una interfaz de usuario altamente personalizada aprovechando las capacidades de interfaz de usuario y de entrada/salida nativas del lenguaje.

Otras interfaces de línea de comandos (*Other command-line interfaces*)

La línea de comandos proporciona una interfaz tanto entre programas como con el usuario. En este sentido, una línea de comandos constituye una alternativa a un cuadro de diálogo. Los editores y las bases de datos presentan una línea de comandos en la que pueden ejecutarse intérpretes de comandos alternativos. Por otro lado, es posible contar con opciones en la línea de comandos que abren un cuadro de diálogo; la versión más reciente de *Take Command* incluye esta característica. dBase utilizaba un cuadro de diálogo para construir líneas de comandos, las cuales podían editarse posteriormente antes de su ejecución.

Programas como BASIC, `diskpart`, Edlin y QBASIC proporcionan interfaces de línea de comandos, algunas de las cuales hacen uso del *shell* del sistema. BASIC está modelado a partir de la interfaz predeterminada para las computadoras Intel de 8 bits. Las calculadoras se pueden ejecutar como interfaces de línea de comandos o mediante cuadros de diálogo.

Emacs proporciona una interfaz de línea de comandos en forma de su *minibuffer*. Los comandos y sus argumentos se pueden ingresar utilizando las funciones estándar de edición de texto de Emacs, y la salida se muestra en un *buffer* diferente.

Existen varios juegos en modo texto, como *Adventure* o *King's Quest I-III*, que dependían de que el usuario escribiera comandos en la parte inferior de la pantalla. El jugador controla al personaje introduciendo órdenes como `get ring` (tomar anillo) o `look` (mirar). El programa devuelve texto que describe lo que ve el personaje o lleva a cabo la acción. La aventura conversacional *The Hitchhiker's Guide to the Galaxy*, una obra de ficción interactiva basada en el libro homónimo de Douglas Adams, es un juego con interfaz de línea de comandos de estilo teletipo.

La más destacada de estas interfaces es la interfaz de flujos estándar (*standard streams*), que permite transferir la salida de un comando hacia la entrada de otro. Los archivos de texto también pueden cumplir cualquiera de estas dos funciones. Esto proporciona las interfaces de canalización (*piping*), filtros y redirección. En Unix, los dispositivos también son archivos, por lo que el tipo de archivo habitual que utiliza el *shell* para `stdin`, `stdout` y `stderr` es un archivo de dispositivo `tty`.

Otra interfaz de línea de comandos permite que un programa *shell* inicie programas auxiliares (*helper programs*), ya sea para abrir documentos o arrancar una aplicación. El comando se procesa internamente en el *shell* y luego se transfiere a otro programa para abrir el documento. La interfaz gráfica de Windows y OS/2 se apoya enormemente en líneas de comandos transmitidas a otros programas —ya sean de consola o gráficos—, los cuales procesan habitualmente la línea de comandos sin presentar una consola de usuario.

Motivación

Aprender a programar es increíblemente gratificante, pero también puede ser difícil y frustrante. Las mayores fortalezas que puedes tener como estudiante son el deseo de construir, una mentalidad orientada a la resolución de problemas y la persistencia frente a los contratiempos.

La industria del desarrollo web tiene una larga historia de desarrolladores exitosos provenientes de los más diversos orígenes, por lo que las personas suelen darle más importancia a lo que realmente has construido que al camino que recorriste para lograrlo.

¿Por qué este curso?

Queremos que sepas que esto no será fácil.

Existen muchos otros planes de estudio en línea para principiantes, pero a menudo se enseñan en un entorno extremadamente aislado y controlado, y abordan únicamente un tema específico.

Este curso adopta una perspectiva realista de lo que necesitas saber y hace que prepares y trabajes en tu propio entorno, de manera muy similar a lo que harás cuando consigas un empleo.

Reconoce que necesitas una gran variedad de habilidades y lenguajes para alcanzar un nivel que te permita ser contratado.

Este curso evoluciona constantemente gracias a personas como tú que avanzan en el plan de estudios y devuelven el favor mejorando progresivamente nuestro contenido con el tiempo.

Una vez que comiences a sentirte cómodo con las herramientas, ¡empieza a contribuir en proyectos de código abierto, como colaborar en este curso! Cuanto más contribuyas y apoyes a traves de nuestra cuenta de Binance Pay que se encuentra en la descripción, más aprenderás sobre lo que eres capaz de hacer y más cerca estarás de ser contratable. ¡Estos proyectos también se verán excelentes en tu currículum!

Conclusión

Esperamos que hayas obtenido una mejor idea de lo que realmente hace un desarrollador web y cómo podría ser tu vida si decidieras asumirlo como carrera. Esto solo ha sido un adelanto del mundo del desarrollo web.

En este curso de «Fundamentos», realizarás un recorrido por todo el espectro de temas que eventualmente necesitarás conocer. Este curso salta entre una variedad de temas que pueden ser totalmente desconocidos para ti, brindándote una pequeña prueba de cada uno para luego continuar.

Los cursos siguientes profundizarán en estos temas. Construirás docenas de scripts, proyectos y sitios web para afianzar esas habilidades que te permitirán conseguir empleo.

Sí, va a ser desafiante.

Pero también va a ser divertido.

Y podría, incluso, cambiarte la vida.

¿Qué estás esperando?

Por qué aprender a programar es tan jodidamente difícil

Lo que todo principiante necesita saber absolutamente sobre el viaje que tiene por delante:

Quincy Larson era solo un «tipo en traje en una oficina» y decidió que quería aprender a programar. Así que empezó a preguntar a su alrededor. Comenzó aprendiendo un poco de Ruby y luego se vio a sí mismo echando un vistazo superficial a otros lenguajes como Scala, Clojure y Go. Aprendió Emacs, luego Vim e incluso la distribución de teclado Dvorak. Aprendió Linux, jugueteó con Lisp y programó en Python mientras vivió en la línea de comandos durante más de medio año.

Como una hoja en un tornado, los consejos que Quincy recibió lo sacudieron primero en una dirección, luego en otra y después en otra, hasta que finalmente completó «cada programa de cursos en línea imaginable». Al final de todo esto, a pesar de haber conseguido finalmente un trabajo de desarrollo de software, Quincy:

> ... estaba convencido de que los programadores aparentemente normales con los que me cruzaba eran en realidad sociópatas que habían experimentado, y luego reprimido, el trauma de aprender a programar.

Auch. ¿Te suena familiar?

Fase I: La luna de miel guiada de la mano

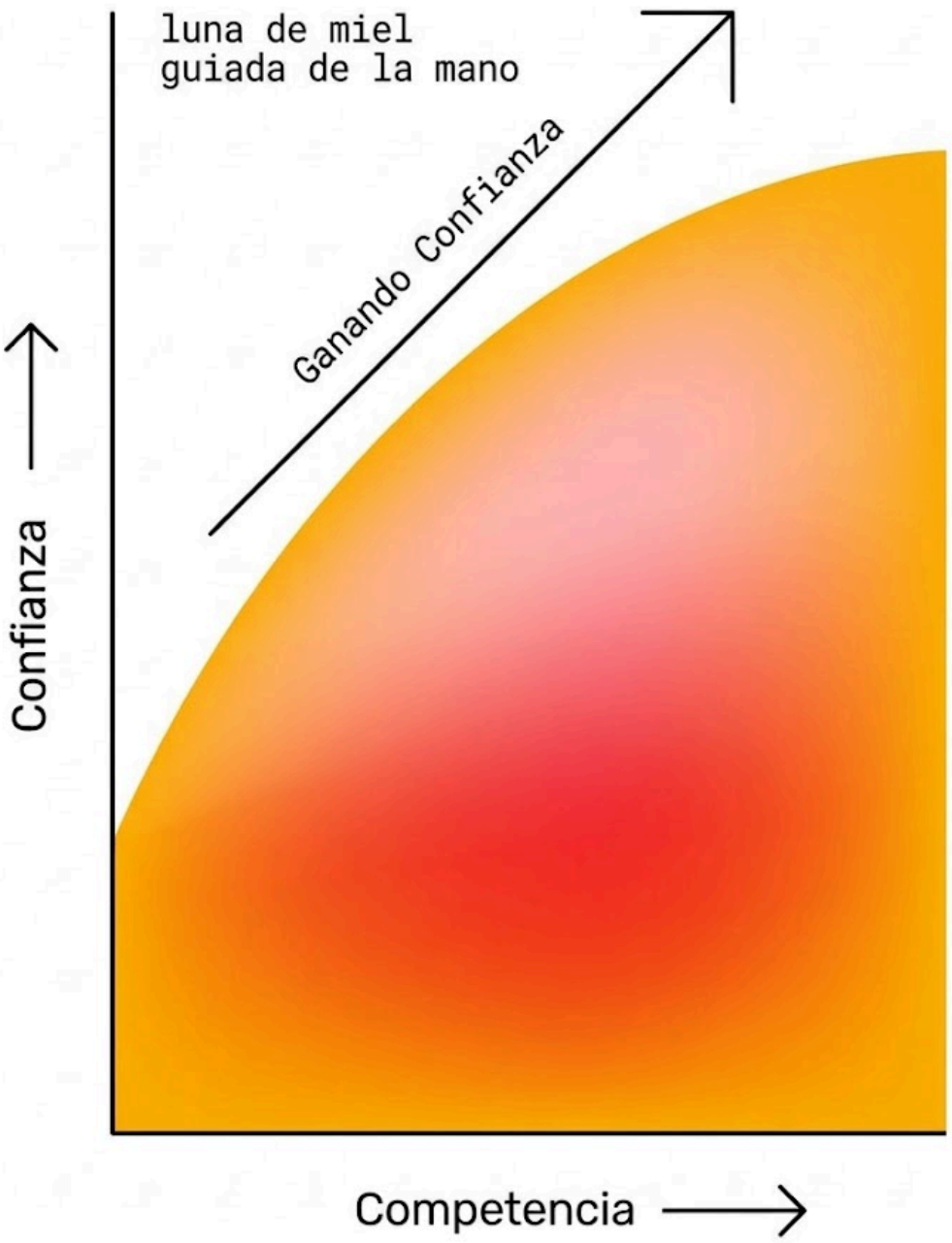
Es realmente difícil culpar a cualquiera por entrar en la industria de la programación con expectativas desmesuradas.

Por un lado, has escuchado rumores de lo difícil que es la programación desde que eras joven, como viejos cuentos de ancianas pensados para asustar a los niños y hacer que estudien ciencias sociales en su lugar.

Por el otro, el movimiento «Aprende a programar» ha hecho un trabajo fantástico derribando barreras y mostrándole a la gente que el código es, en realidad, bastante inofensivo. Herramientas como Codecademy y Treehouse te tienden la mano con la mayor de las delicadezas para asegurarte que tú también (¡es más, cualquiera!) no solo puedes aprender a programar, sino también convertirte en un desarrollador hecho y derecho.

De repente, el problema ya no es el miedo, sino la sobreabundancia de esperanzas y altas expectativas.

Y, en su mayor parte, estas herramientas introductorias hacen un trabajo excelente guiándote como a un niño en un paso peatonal, dejando atrás las grandes y aterradoras variables y declaraciones condicionales a través de las primeras fases de la sintaxis de programación. A medida que conquistas uno tras otro de sus desafíos gamificados, tu confianza aumenta. ¡Tal vez sí puedes hacer esto después de todo! ¿Qué tan difícil puede ser? ¡Básicamente ya eres un desarrollador!



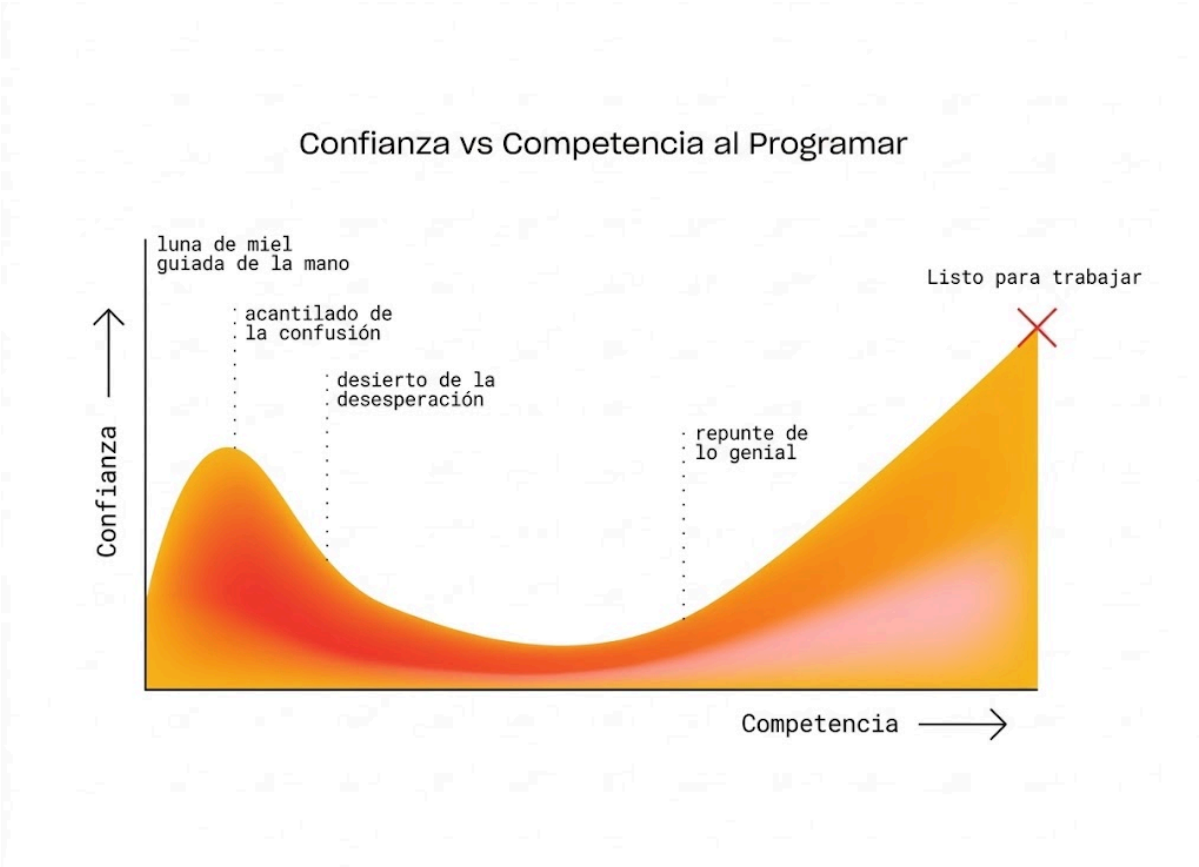
Aquí está el problema: estás en lo que me gusta llamar la fase de la «Luna de miel guiada de la mano». Aunque sientas que el final está a la vuelta de la esquina, solo estás en una fracción del camino. Esto es apenas el comienzo…

Trazando el camino a seguir

Antes de sumergirnos en la Fase II, echemos un vistazo al panorama general.

En esta publicación, te guiaré a través de las cuatro fases del viaje típico hacia la programación y lo que tendrás que hacer para sobrevivir a cada una de ellas. También verás cómo dos factores clave —la densidad de recursos y el alcance del conocimiento requerido— definen este viaje.

La trayectoria hacia estar listo para un trabajo se puede graficar en función de cómo cambia tu nivel de confianza a medida que aumenta tu capacidad:



Esta es una relación relevante porque tu confianza está fuertemente correlacionada con tu felicidad y porque el punto donde coinciden tu confianza y tus capacidades es el mejor indicador que tengo para ese punto ideal en el que estás oficialmente «listo para trabajar».

Analizaremos los desafíos únicos de las 3 fases restantes en un momento, pero esto es esencialmente lo que implica cada una de ellas:

1. **La luna de miel guiada de la mano** es ese paseo lleno de alegría a través de recursos altamente pulidos que te enseñan cosas que parecen difíciles, pero que son totalmente realizables gracias a su apoyo intensivo. Aprenderás principalmente la sintaxis básica, pero te sentirás genial con tus logros.
2. **El acantilado de la confusión** es la dolorosa constatación de que todo es mucho más difícil cuando se acaba el acompañamiento guiado, y sientes que realmente aún no puedes hacer nada por tu cuenta. Tus principales desafíos son la depuración

constante de errores y el no saber muy bien cómo hacer las preguntas correctas mientras luchas por conseguir cualquier tipo de impulso.

3. **El desierto de la desesperación** es el largo y solitario viaje a través de un paisaje sin caminos donde cada nueva dirección parece correcta, pero con frecuencia estás dando vueltas en círculos y te encuentras hambriento de recursos que te ayuden a salir adelante. Cuidado con los «espejismos de la manía», que como sirenas del desierto, te desviarán del camino.
4. **El repunte de lo genial** es cuando finalmente has encontrado un camino a través del desierto y has logrado comprender cómo construir aplicaciones. Sin embargo, tu código todavía está aislado y es frágil como un castillo de naipes. Ganas confianza porque tus sitios parecen funcionar, has dominado algunos patrones útiles y tus amigos piensan que tus interfaces son geniales, pero te aterroriza mirar bajo el capó y, en última instancia, no sabes cómo llegar a un código listo para producción. ¿Cómo cierras esa brecha para conseguir un trabajo real?

He entrevistado a cientos de aspirantes a desarrolladores a lo largo de los últimos años y he escuchado los ecos de la misma historia una y otra vez. Mi objetivo con esta publicación es que abordes el viaje del estudiante con ambos ojos bien abiertos y con un plan lo suficientemente claro como para evitar las trampas comunes de quienes vinieron antes que tú.

Volvamos a la Fase II...

Fase II: El acantilado de la confusión

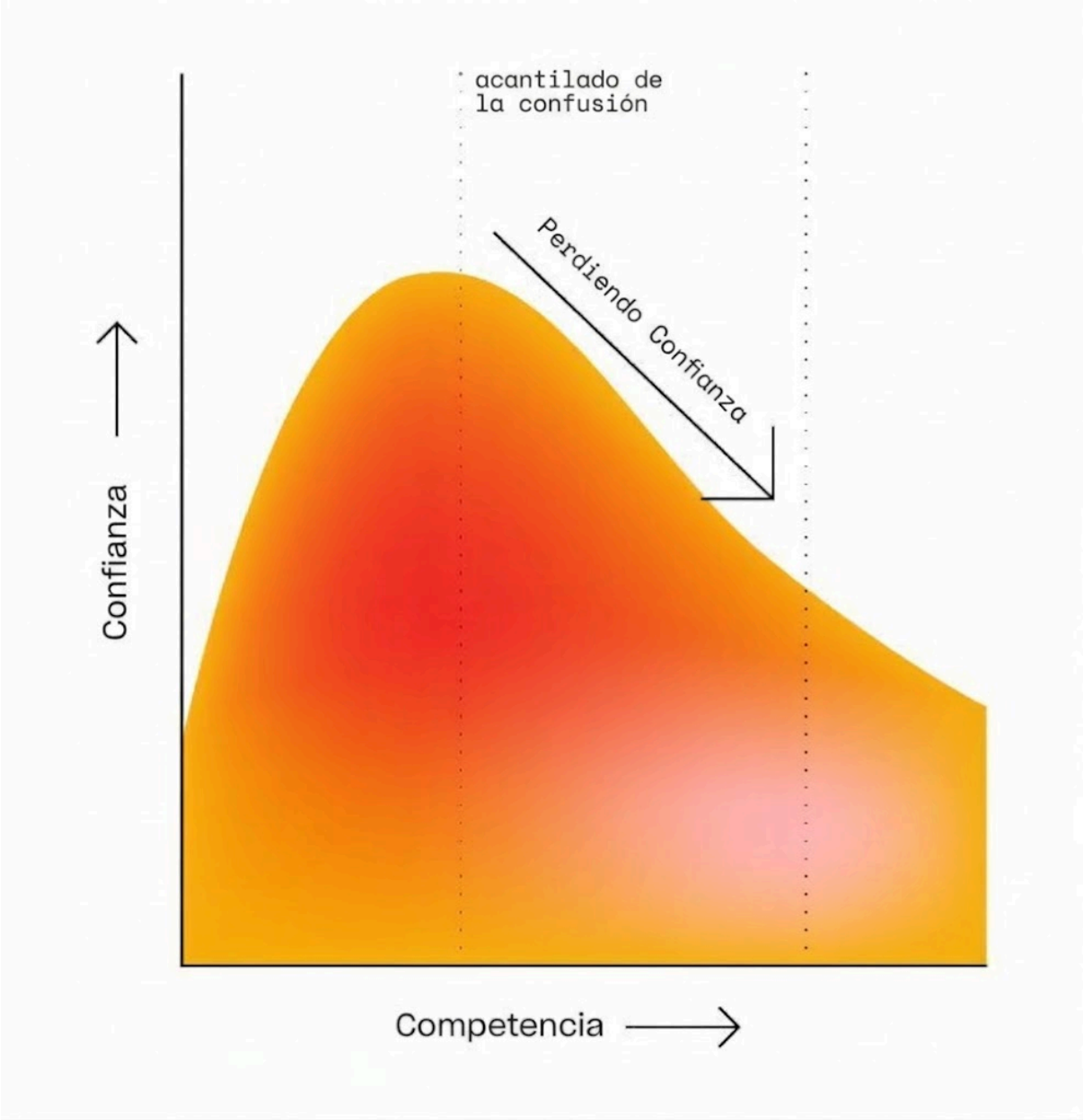
Así que estás en la Fase I —la «Luna de miel guiada de la mano»— acumulando medallas y completando desafíos de programación mientras tu confianza y tus capacidades crecen. Esto no está tan mal... ¿a qué viene tanto alboroto? Has llegado a la «Cima de la exuberancia irracional»...

¡Ten cuidado! Estás a punto de dar un paso en falso sobre un precipicio que ha destrozado a muchos estudiantes apasionados y los ha relegado al bando de «programar es demasiado difícil». El momento preciso en que ocurre este salto es la primera vez que te sientas frente al teclado, abres tu editor de texto e intentas construir un proyecto desde cero sin ninguno de esos elegantes editores integrados en el navegador, código preestructurado o pistas útiles.

Mierda.

Podrías estirar esto un poco más siguiendo tutoriales, pero nadie ha alcanzado jamás el cielo sin despegar del suelo y, en algún momento, vas a tener que crear magia a partir de

un archivo de texto en blanco. Acabas de entrar en la segunda fase del aprendizaje, donde la confianza se desploma estrepitosamente contra la tierra: el «Acantilado de la confusión».



Así que empiezas a construir. Luches y arañas tu camino hacia una solución apenas funcional, pero algo falta. Estás en una guerra contra los errores (*bugs*) que hace que *Starship Troopers* parezca algo inofensivo. Sientes que cada victoria se logró únicamente por un golpe de suerte al buscar en Google, y tu confianza en que algún día podrás entender estas cosas se desploma.

Esta es una fase particularmente frustrante de presenciar como educador y para todos los participantes de nuestra industria. Puede que la programación no sea perfecta para todos, pero queremos que progreses porque, a veces, las historias más impensables se convierten en los éxitos más grandes.

Cuando el acompañamiento guiado termina, y a los estudiantes se les empuja fuera del acantilado y se les dice que vuelen, demasiadas personas potencialmente increíbles terminan en una espiral hacia las rocas de la frustración sin haber aprendido a aletear.

La parte aterradora es que ni siquiera has llegado a lo más sustancioso todavía. Esta segunda fase, el «Acantilado de la confusión», es aún muy temprana. Una vez que finalmente hayas aplastado suficientes errores (*bugs*) como para poner fin a la octava plaga de Egipto y hayas terminado realmente un par de proyectos —marcando así el final de la Fase II—, todavía estás empezando.

Para aquellos que verdaderamente están listos para hacer de esto una carrera, sobrevivir al Acantilado de la confusión es a menudo el punto en el que decides apostarlo todo por tu nueva vida. Pero demasiados se quedan atrás. Y, desafortunadamente, estás a punto de entrar en el «Desierto de la desesperación».

Los dos factores clave en juego

Entonces, ¿qué marca realmente la diferencia entre una fase y la siguiente? ¿Por qué la Fase II (el Acantilado de la confusión) fue tan terrible en comparación con la Fase I (la Luna de miel guiada de la mano)? Entender esto te ayudará a darte cuenta de que no es en absoluto tu culpa si tu viaje se parece a lo que acabamos de describir.

Básicamente, hay dos fuerzas clave en juego en cada fase: la densidad de recursos y el alcance del conocimiento. Veamos cuáles son antes de explorar cómo definen la Fase III.

Factor 1: La densidad de recursos

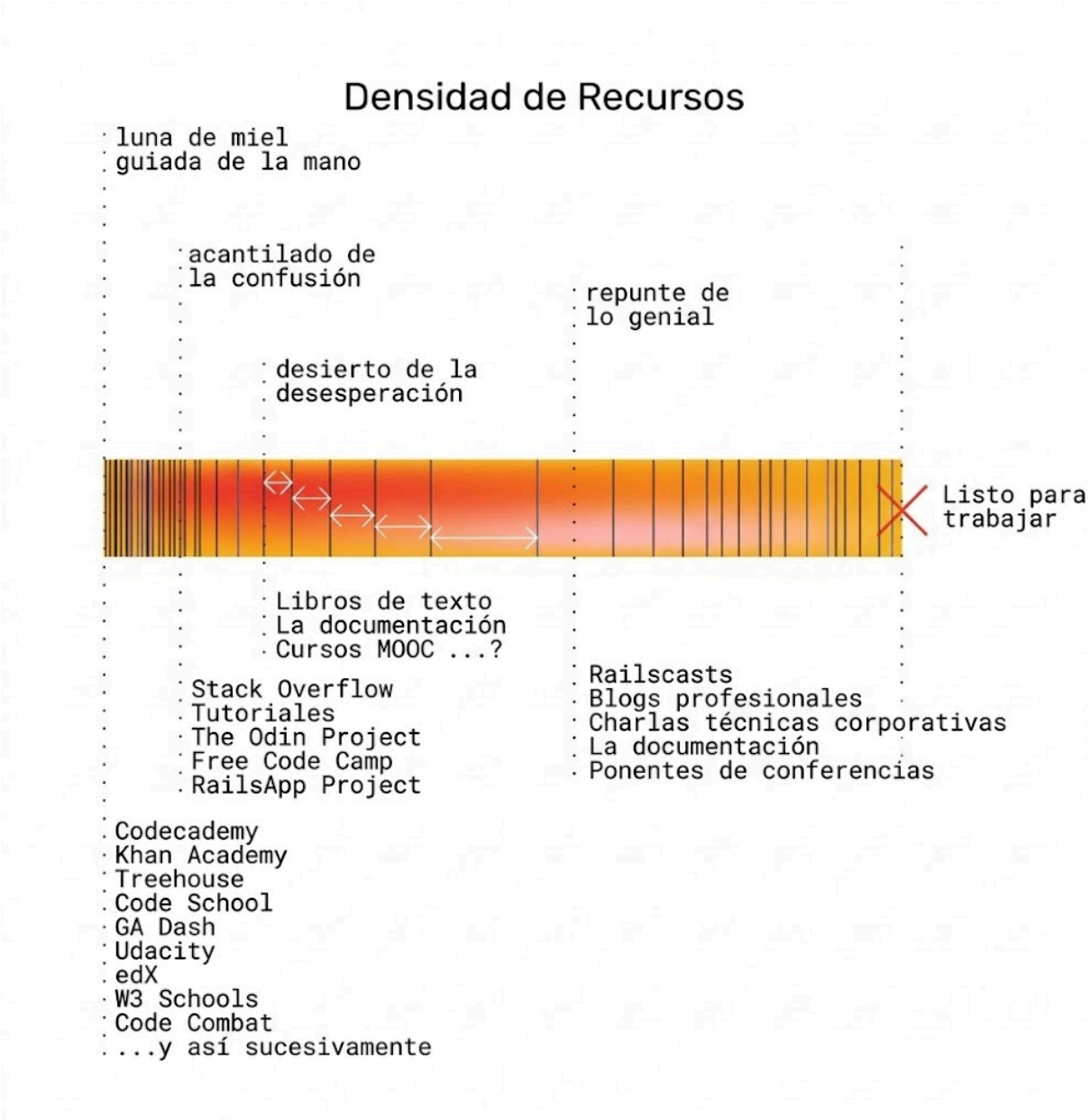
Como dije anteriormente, cuando estás empezando, parece que hay un millón de recursos allá afuera intentando llevarte de la mano y atraerte a la programación. ¡Eso se debe a que realmente los hay!

Si buscas «Aprender a programar», te encontrarás con una pared de herramientas, textos, videos y tutoriales útiles y serviciales. Y, francamente, ¡son geniales! Nunca antes habían existido tantas formas de empezar a aprender a programar.

Desafortunadamente, en las fases posteriores, la densidad de recursos cae rápidamente. Cualquiera que haya dado el salto de principiante a intermedio puede dar fe de que hay una GRAN diferencia entre la cantidad de recursos disponibles cuando estás empezando y cuando buscas ayuda por primera vez para construir cosas por tu cuenta sin demasiado acompañamiento guiado.

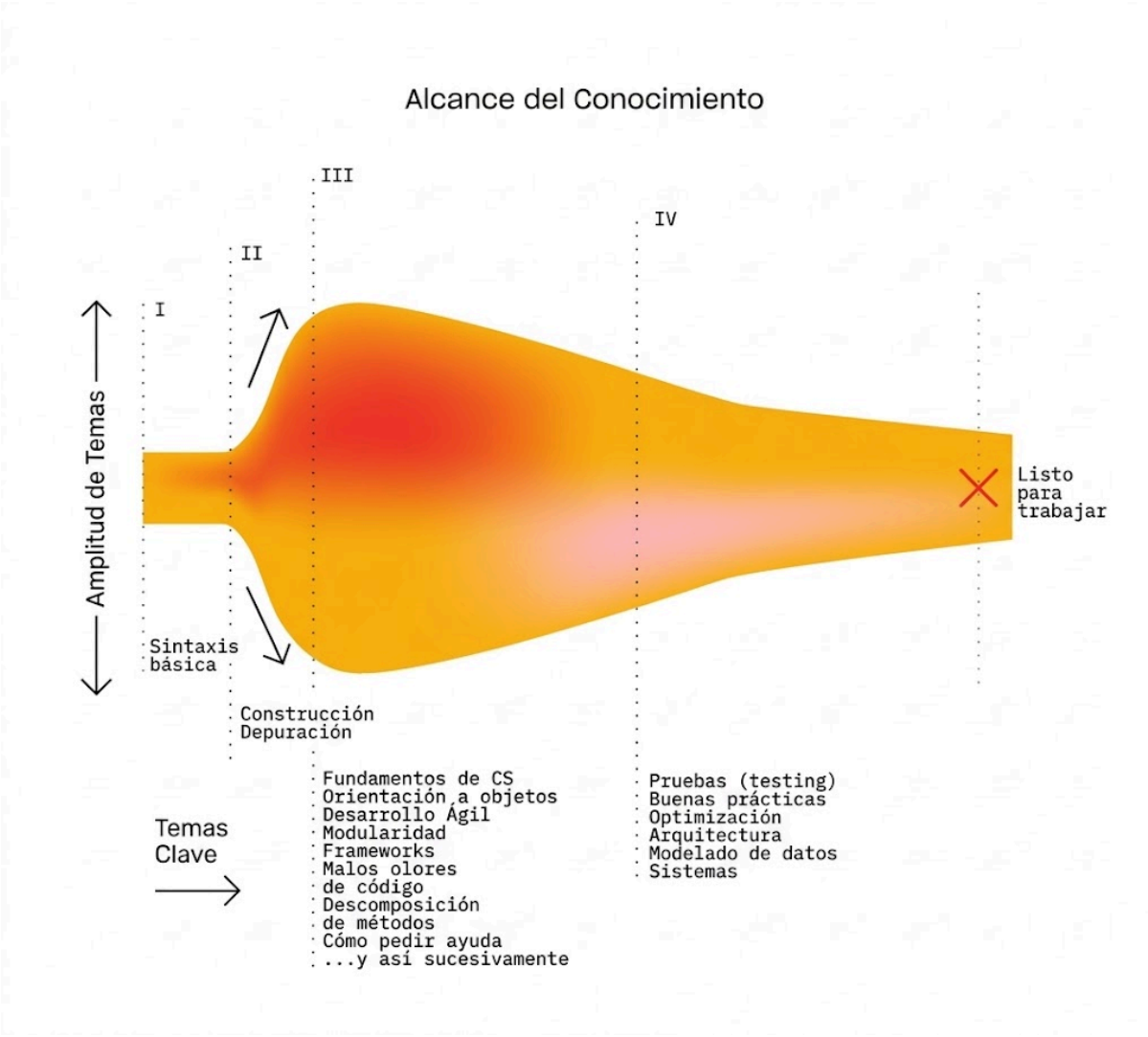
Este problema se exacerba a medida que la cantidad de conocimiento aumenta rápidamente al entrar en la Fase III, y es una de las razones por las cuales llamamos a esa etapa el «Desierto de la desesperación». Una vez que superas esto y comienzas a sentirte cómodo con qué buscar exactamente, los recursos regresan y eres capaz de trabajar con herramientas más técnicas como blogs de la industria y transmisiones en pantalla (*screencasts*). Parte de esto consiste simplemente en entender qué preguntas hacer.

Así es como se ve la densidad de recursos en cada fase (una mayor densidad de líneas indica más recursos):



Factor 2: El alcance del conocimiento

Ahora hablemos de un problema relacionado: el alcance del conocimiento. Esto representa la amplitud total de nuevos temas que necesitas aprender en cada fase. Así es como se ve:



Cuando comienzas a aprender, el conjunto de cosas que necesitas entender es reducido. Todos, independientemente de sus objetivos, lenguaje o antecedente, necesitan descubrir qué es un bucle *for*, cómo construir lógica condicional y otras estructuras básicas de la sintaxis de programación. En última instancia, ni siquiera existen tantos de estos conceptos fundamentales, por lo que el alcance del conocimiento durante esa fase es muy estrecho.

Tan pronto como te alejas de lo básico, notas una rápida ampliación del alcance del conocimiento, ya que debes comenzar a adquirir conocimientos más complejos, como comprender los errores y saber cuándo utilizar el código que ya sabes emplear. Esto es diferente porque no existe una respuesta «correcta» para una pregunta clara... las cosas se vuelven difusas.

Cuando avanzas hacia la tercera fase, el alcance del conocimiento se expande desmesuradamente. Ahora necesitas comprender qué herramientas utilizar, qué lenguajes aprender, los fundamentos subyacentes de la ciencia de la computación, cómo escribir código modular, orientación a objetos, buen estilo y cómo pedir ayuda (por nombrar solo

algunos). Cada visita a Google o Hacker News te lleva por un nuevo conjunto de madrigueras de conejo y te abruma con más cosas que no sabes, pero que sientes que deberías saber.

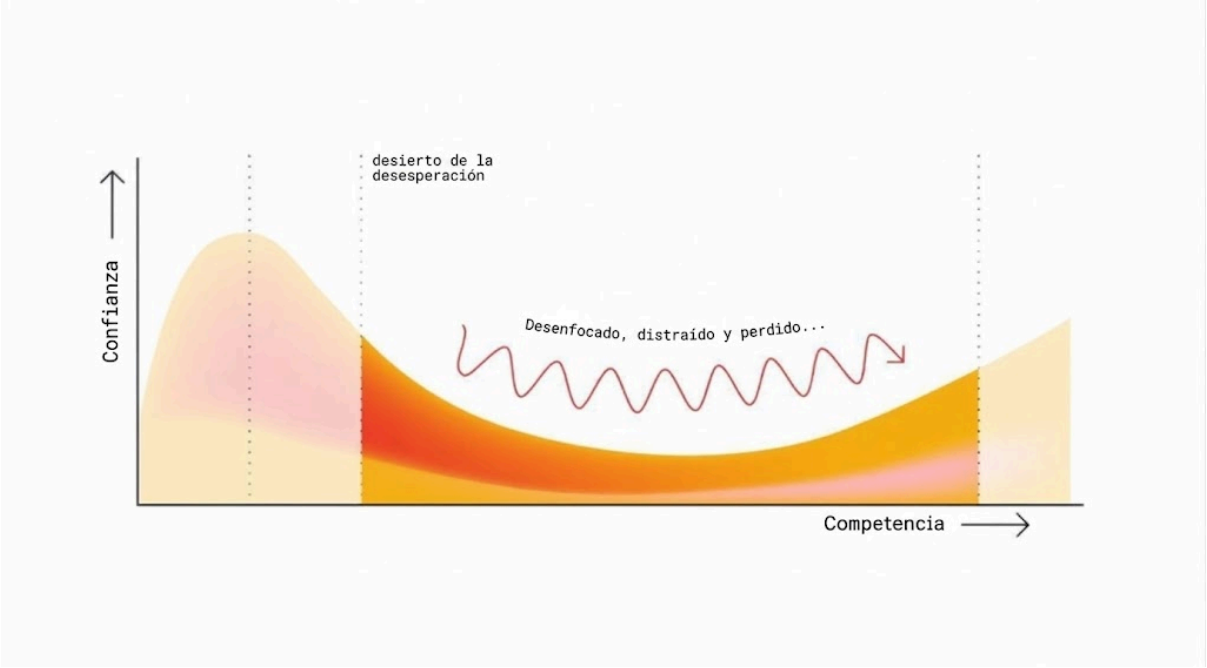
No sabes lo que no sabes.

Solo cuando finalmente has encontrado algo de firmeza y has dejado el desierto, el alcance comienza a estrecharse de nuevo. Para ese momento, has encontrado la tecnología que elegiste y su lugar en el ecosistema. Finalmente (en su mayor parte) sabes lo que no sabes y puedes trazar un camino a través de ello. Continuarás aumentando tu enfoque a medida que sigas avanzando y te adentres en el comienzo de tu carrera.

Fase III: El desierto de la desesperación

Con una comprensión de estos factores, puedes ver que el Acantilado de la confusión es realmente solo un punto de inflexión. El dolor causado por la combinación tóxica de un alcance del conocimiento en rápido aumento y una densidad de recursos en caída da como resultado lo que yo llamo el «Desierto de la desesperación».

En esencia, este desierto es donde sabes que hay un final en alguna parte, pero no sabes cómo llegar allí:



El desierto es largo y está lleno de peligros. A lo largo del camino te sentirás atraído por los «Espejismos de la Manía»: docenas de recursos tentadores que parecen tener las

soluciones que estás buscando, pero que te dejarán, una vez más, en un lugar donde la arena solitaria se extiende hasta la línea de cada horizonte.

Tal vez te inscribas en un par de cursos MOOC de Coursera, Udacity o edX. O encuentres un tutorial que pretenda llevarte hasta el final. Pensaste que habías aprendido las lecciones de la Luna de Miel Guiada de la Mano —que no hay respuestas fáciles—, pero la tentación de buscar la salvación es demasiado grande y caes en la promesa de que este sí te llevará a la meta donde los demás no lo lograron.

No puedes aprender estas cosas en una semana, ni en un mes, ni en una sola clase universitaria, no importa lo que diga cualquiera, ¡así que deja de caer en eso!

Hay MUCHO más que aprender de lo que probablemente esperabas. Incluso si logras poner en marcha algunas aplicaciones, es difícil no sentirse perdido en el panorama general de convertirte en un verdadero profesional. Es difícil medir tu progreso. ¿Cómo sabes qué necesitas aprender o si siquiera estás aprendiendo las cosas correctas?

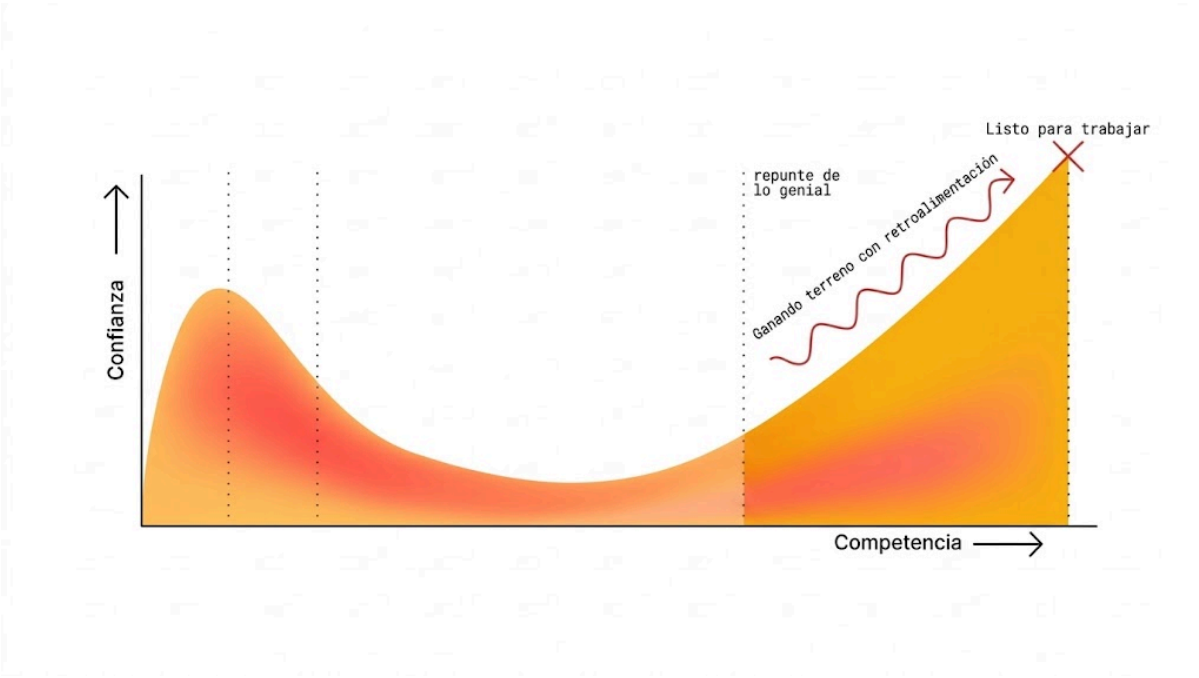
Incluso si estás apuntando en la dirección correcta, es difícil medir tu avance. Es posible que te sientas totalmente perdido hasta el mismísimo momento en que finalmente seas capaz de construir algo que se vea y funcione de la manera que esperabas. Sin embargo, con suficiente perseverancia y una buena brújula, con el tiempo lograrás lanzar tus primeros proyectos «reales» y te darás cuenta de que finalmente estás empezando a entenderlo.

Seguro que ha sido difícil hasta ahora, pero tal vez esto del desarrollo web no sea tan malo después de todo... ¡Todo marcha de maravilla! *(Everything's coming up Milhouse!)*

Fase IV: El repunte de lo genial

Has logrado atravesar el desierto y tu confianza está creciendo. Tu dominio del *Google-fu* es excelente y finalmente eres capaz de entender esas detalladas publicaciones de blogs de la industria y transmisiones en pantalla (*screencasts*). Tal vez te has profundizado en un lenguaje o *framework* en particular y tienes la confianza de que puedes construir y lanzar una aplicación funcional.

Este es el «Repunte de lo genial»:



Todo puede parecer ir bien hacia afuera, pero en el fondo sabes que aún no has llegado.

Puedes hacer que esa aplicación funcione, ¿pero qué está pasando debajo de la superficie? Tu código está sostenido con cinta adhesiva y pabilo y, lo peor de todo, ni siquiera sabes qué partes son terribles y cuáles están realmente bien. Tus chispazos periódicos de brillantez se ven contrarrestados por errores de novato y, peor aún, por la sospecha creciente de que todavía no tienes ni la más remota idea de lo que estás haciendo.

Esta es una fase bipolar. Sientes que la mitad de ti es un desarrollador a prueba de balas y la otra mitad es una delgada capa de efectividad que cubre a un novato desconcertado que está metido en esto hasta el cuello. Cuanto más avanzas, más crece una carcomedora sensación de incertidumbre de que alguien te vaya a «descubrir» como un fraude.

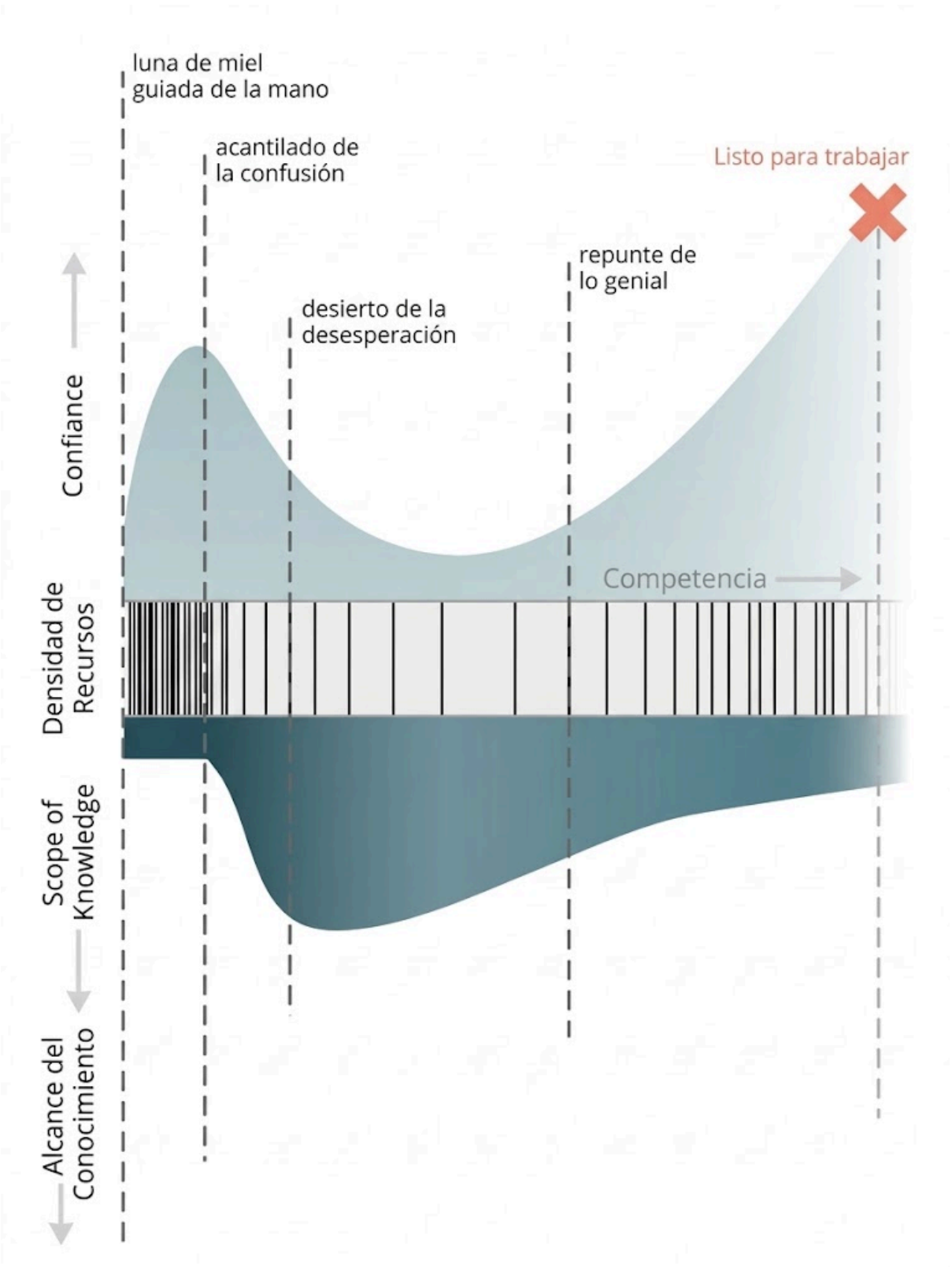
Sientes que ya deberías ser un desarrollador, pero la distancia entre el código que estás escribiendo y un entorno de trabajo «profesional» no podría sentirse más lejana...

Sin embargo, con el tiempo lo lograrás. ¡Hay demasiado impulso como para no hacerlo! El Desierto de la Desesperación quedó atrás y el Acantilado de la Confusión es un recuerdo distante. Finalmente, verdaderamente, estás en el repunte. Estás aprendiendo más rápido y de manera más inteligente que nunca y, con el tiempo, habrás absorbido suficientes buenas prácticas como para que tu conocimiento lleno de lagunas se consolide en un conjunto de habilidades de nivel de producción.

El Repunte de lo Genial siempre toma más tiempo del que esperas y se siente interminable porque estás muy cerca... pero llegarás allí. Si eres lo suficientemente persistente de las formas correctas (un tema seguro para una futura publicación), convencerás a alguien de que te pague por seguir aprendiendo. El trabajo es tuyo.

Cómo se ve todo en conjunto

Así que ahora has visto el camino que tienes por delante y las razones por las que puede ser difícil. Cuando combinas las cuatro fases que acabamos de cubrir con los factores que las definen, el resultado se parece bastante al siguiente gráfico:

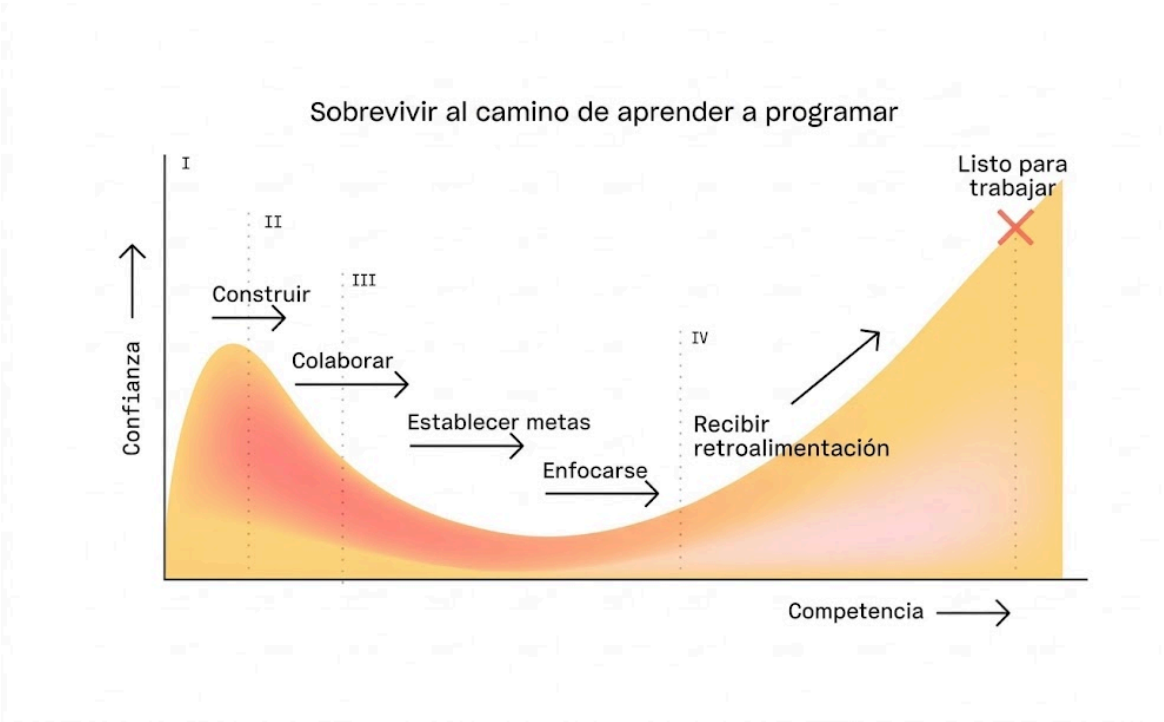


Una cosa es conocer el camino y otra muy distinta es recorrerlo. Empecemos con el pie derecho.

Cómo salir con vida

El viaje parece intenso y, francamente, a menudo lo es. Es importante que entiendas a qué te enfrentas, particularmente si lo haces por tu cuenta. Pero no tienes que hacerlo solo. Hay formas de atajar la mayoría de estos problemas. Aprender a programar raras veces es tan fácil como la gente lo hace ver, pero tampoco suele ser tan difícil como parece en lo más profundo de tu desesperación.

En esta sección, presentaré las tácticas clave que puedes utilizar para mantenerte orientado en la dirección correcta.



Cómo sobrevivir a la "luna de miel guiada"

La enorme cantidad de recursos disponibles en la "luna de miel guiada" (*Hand-Holding Honeymoon*) hace que esta etapa sea muy divertida. Hacen un excelente trabajo al introducirte gradualmente en el tipo de pensamiento lógico que necesitarás desarrollar en las siguientes fases. Es un gran momento para empezar a aprender a programar, así que intenta disfrutarlo y ten en cuenta estos dos consejos:

1. **Prueba diferentes recursos al principio** para descubrir cómo aprendes mejor y qué tipo de proyectos te resultan más interesantes. Tal vez sean los desafíos rápidos de Khan Academy, los ejercicios en el navegador de Codecademy o la experiencia de mentoría personalizada de Thinkful. Mantén la mente abierta al inicio e ignora cualquier comentario sobre lo que "deberías" aprender... en esta fase, todo el código es igual.
2. **Elige un solo recurso y mantente firme** una vez que hayas encontrado el que mejor se adapta a ti. Avanza hasta el final de su curso de introducción, lo cual debería darte todos los conocimientos fundamentales que necesitas para escribir aplicaciones y scripts básicos. Después de eso, prepárate para empezar a construir por tu cuenta.

Cómo sobrevivir al "acantilado de la confusión"

Casi todo el mundo pasa por el "acantilado de la confusión" (*Cliff of Confusion*) porque la única forma de convertirse en desarrollador es, bueno, desarrollando. Puedes fingir que estás construyendo algo al registrarte en tutoriales (o en tutoriales que se disfrazan de cursos "completos"), pero solo estás posponiendo lo inevitable. Los tutoriales son una buena forma de hacer la transición desde las ofertas introductorias más guiadas, pero en algún momento tendrás que dejar el chupón (o chupete) y enfrentarte al mundo real.

Tres consejos para hacer la transición hacia la construcción por tu cuenta:

1. **Trabaja con alguien más,** incluso si es otro principiante. Te sorprenderá lo mucho más fácil que es depurar (*debug*) un error imposible cuando se comparten dos pares de ojos.
2. **Lee el código de otras personas** para familiarizarte con las buenas prácticas y patrones. Intenta comprender por qué el autor hizo lo que hizo. No intentarías convertirte en novelista sin leer libros también, ¿verdad? Nos enfocaremos en esto en una próxima publicación pero, por ahora, mantente atento a cualquier pequeño problema o proyecto para el cual otras personas ya hayan escrito soluciones.
3. **Empieza poco a poco y construye constantemente.** Deberías tener en mente proyectos grandes e interesantes para el futuro, pero primero necesitas sentirte cómodo depurando errores y buscando recursos mediante desafíos pequeños. Realmente no hay sustituto para la experiencia.

Cómo sobrevivir al "desierto de la desesperación"

Una vez que te sientes cómodo depurando errores, tu mayor problema pasa a ser el torrente abrumador de conocimientos necesarios y una total pérdida de rumbo sobre cómo aprenderlo todo... el "desierto de la desesperación" (*Desert of Despair*). En este punto, lo que realmente necesitas es un camino firme hacia adelante. Los "espejismos de la obsesión" (*Mirages of Mania*) representan todos esos caminos secundarios interesantes, madrigueras de conejo y esquemas para "volverse experto en poco tiempo" que, al final, solo te hacen perder el tiempo.

Por lo tanto, las claves para salir del desierto de la desesperación son:

1. **Ten una meta clara de lo que quieres lograr,** porque de lo contrario terminarás dando vueltas en círculos aprendiendo todo tipo de cosas interesantes pero, a fin de cuentas, improductivas. Si te sobra el tiempo, por supuesto, ignora esto...
2. **Encuentra un camino sólido que te lleve directo a la meta que te has propuesto** y verifica que realmente te llevará allí. Aquí es donde debes mirar más allá de los eslóganes de marketing y las caras sonrientes en los sitios web de los cursos o en las portadas de los libros para preguntarte: "¿Esto me ayudará a lograr la meta que me propuse o no?".
3. **Enfócate y evita las distracciones** porque, si eres el tipo de persona a la que le interesa aprender a programar, también eres el tipo de persona que se interesa por aprender cualquier otra cosa genial. Cuando la programación se ponga difícil, debes ser capaz de seguir adelante en lugar de simplemente ir a probar la siguiente novedad que llame tu atención.

Si logras identificar un camino y mantenerte firme en él, tarde o temprano avanzarás a la siguiente fase en lugar de pasar meses o años persiguiendo espejismos entre las arenas movedizas de este desierto.

Cómo sobrevivir al "repunte del optimismo"

El "repunte del optimismo" (*Upswing of Awesome*) es una de las transiciones más engañosas. Ya puedes desarrollar aplicaciones, pero lo que realmente quieres es convertirte en un desarrollador web profesional. Superar esta fase y conseguir un empleo requiere que hagas tres cosas:

- **Busca y sigue las mejores prácticas de programación.** Necesitas entender la diferencia entre una solución cualquiera y la mejor solución posible. Las buenas prácticas marcan la gran diferencia entre programar por tu cuenta a base de parches (*hacking*) y escribir código con calidad de producción en un entorno laboral real.
- **Cuestiona tus suposiciones** porque es muy probable que hayas avanzado dejando grandes vacíos de conocimiento sin siquiera darte cuenta. Necesitas diagnosticar y reparar esos baches.
- **Domina las habilidades menos atractivas,** esas que rara vez se mencionan pero que son cruciales para la transición al entorno profesional. Esto incluye temas como pruebas (*testing*), modelado de datos, arquitectura y despliegue (*deployment*); herramientas que son fáciles de pasar por alto pero que resultan totalmente fundamentales para un buen desarrollo.

La clave para lograr todo esto y superar con éxito el repunte del optimismo es **recibir retroalimentación**. Los estudiantes que han aprendido de forma totalmente autodidacta pueden ser productivos, pero rara vez logran un código lo suficientemente legible, modular y mantenible como para resultar atractivos en el mercado profesional. Necesitas trabajar con otras personas que desafíen tus suposiciones, te hagan preguntas directas y profundas, y te obliguen a tapar las grietas de tus conocimientos.

Entonces... ¿se puede lograr?

Todo esto puede sonar abrumador, pero te prometo que muchos otros han perseverado y sobrevivido a este viaje antes que tú. Al comprender el camino que tienes por delante, ya te encuentras en una excelente posición para afrontarlo con un plan enfocado y acceso al tipo de ayuda adecuado.

Obviamente, en esta publicación en particular no hay espacio para profundizar tanto como nos gustaría en cada fase del viaje, ni para ofrecer el tipo de consejos

detallados paso a paso que te mereces. Dicho esto, este es un camino con el que estamos muy familiarizados y que nos apasiona enormemente, por lo que queremos ayudarte en todo lo que podamos.

Descifrando 3 carreras de desarrollo web: Front-End frente a Back-End frente a Full Stack

Abres una nueva pestaña en el navegador, escribes una URL y presionas enter. El sitio web se carga al instante. El diseño impecable, las páginas bien estructuradas y los elementos visuales impresionantes casi te dejan sin aliento.

¿Quién es el responsable de cada parte de esa experiencia? Los desarrolladores web.

A partir de 2024, internet alberga más de 1100 millones de sitios web, y la cifra sigue aumentando. Esto se traduce en una enorme estabilidad laboral para los desarrolladores web, las personas encargadas de programar, construir, analizar y mantener todos esos sitios.

Los sitios web son fundamentales para que cualquier empresa se mantenga competitiva. Dado que las tendencias y las mejores prácticas del desarrollo web evolucionan rápidamente, nunca falta trabajo para los desarrolladores.

Pero, ¿cómo decides qué tipo de desarrollo web elegir? Si has explorado ofertas de empleo o has buscado cursos en línea, probablemente habrás notado que el desarrollo web se divide principalmente en tres categorías: desarrollador front-end, desarrollador back-end y desarrollador full-stack.

Si ya has dado tus primeros pasos con HTML, JavaScript o tal vez Python, pero no estás seguro de qué camino tomar, esta práctica guía es para ti.

¿Qué es un desarrollador front-end?

El front-end de un sitio web es la parte con la que interactúan los usuarios. Todo lo que ves al navegar por la web —desde las fuentes y los colores hasta los menús desplegables y las barras de desplazamiento (*sliders*)— es una combinación de

HTML, CSS y JavaScript controlada por el navegador de tu computadora. Estos elementos se unen para crear la experiencia fluida y sin interrupciones que los usuarios esperan al visitar un sitio web moderno.

Los desarrolladores front-end son los arquitectos de esta experiencia visual e interactiva. Se encargan de transformar los bocetos de diseño (*mockups*) y las estructuras alámbricas (*wireframes*) en páginas web funcionales que atraigan a los usuarios y ofrezcan una navegación intuitiva. Su trabajo impacta directamente en cómo se percibe una marca en línea, lo que convierte a su rol en una pieza fundamental dentro de la estrategia digital de una empresa.

Habilidades y herramientas necesarias para el desarrollo front-end

Los desarrolladores front-end son responsables del código que ve el usuario en un sitio web y de la arquitectura del diseño de su experiencia de usuario inmersiva. Para lograr estos objetivos, deben dominar la programación en HTML, CSS y JavaScript.

Además de dominar estos lenguajes, los desarrolladores front-end necesitan ser competentes en bibliotecas y marcos de trabajo (*frameworks*) web como Bootstrap, Foundation, AngularJS, ReactJS y EmberJS. Estas herramientas aceleran significativamente el desarrollo al proporcionar estructuras y componentes preconstruidos. Por ejemplo, Bootstrap destaca en la creación de diseños adaptables o responsivos (*responsive layouts*), mientras que ReactJS es reconocido por construir interfaces de usuario dinámicas y de alto rendimiento.

Node.js es una herramienta poderosa para los desarrolladores front-end que se utiliza principalmente para administrar dependencias, ejecutar procesos de compilación (*build processes*) y automatizar tareas. Esta optimiza el flujo de trabajo de desarrollo cuando se utilizan *frameworks* como ReactJS o AngularJS.

Al utilizar eficazmente estas tecnologías, los desarrolladores pueden crear experiencias de usuario excepcionales en una gran variedad de dispositivos y plataformas.

Muchas ofertas de empleo para desarrolladores front-end también solicitan experiencia con Ajax (*Asynchronous JavaScript And XML*), una técnica muy utilizada que permite que las páginas se carguen de forma dinámica al descargar datos del servidor en segundo plano. Esta capacidad es crucial para los sitios web modernos, ya que permite interacciones más fluidas y rápidas sin necesidad de recargar la página por completo.

Los desarrolladores front-end trabajan en estrecha colaboración con diseñadores y analistas de experiencia de usuario para llevar los bocetos desde la fase de desarrollo hasta la entrega final. Necesitan asociarse con fluidez con otros equipos de la empresa para comprender los objetivos, necesidades y oportunidades específicas, y luego ejecutar esas directrices. Esto requiere sólidas habilidades de comunicación y una comprensión profunda del comportamiento del usuario y de los principios de diseño.

Más allá de las habilidades técnicas, los desarrolladores front-end exitosos deben tener un gran ojo para el detalle y pasión por la creatividad. A menudo se les encomienda la tarea de garantizar que un sitio web no solo funcione bien, sino que también sea visualmente atractivo. Esto implica seleccionar las fuentes, esquemas de color y diseños adecuados que se alineen con la identidad de la marca, al mismo tiempo que ofrecen una experiencia de usuario positiva.

"Es mucha responsabilidad, pero puede ser muy gratificante", afirma Mikey Ilagan, un desarrollador front-end con ocho años de experiencia. "Soy una persona técnica, pero también visual. Manipular mediante etiquetas y código lo que vemos e interactuamos en las plataformas digitales fue algo que se me dio de forma natural. Me encanta tener un impacto en la interfaz de usuario: los aspectos de una aplicación o sitio web con los que el usuario interactúa y ve".

¿Qué es un desarrollador Back-End?

Entonces, ¿qué es lo que impulsa el front-end de un sitio web? ¿Dónde se almacenan todos esos datos? Aquí es donde entra en juego el back-end. El back-end de un sitio web consta de un servidor, una aplicación y una base de datos. Juntos, estos elementos hacen posible la parte del sitio web que ve el usuario, encargándose de todo, desde el almacenamiento de datos hasta la lógica de negocio.

Un desarrollador back-end construye y mantiene la tecnología que impulsa estos componentes, permitiendo en primer lugar que la interfaz del usuario exista. Son la fuerza invisible detrás de un sitio web, asegurando que todo funcione de manera fluida y eficiente.

Habilidades y herramientas necesarias para el desarrollo Back-End

Para lograr que el servidor, la aplicación y la base de datos se comuniquen entre sí, los desarrolladores back-end utilizan lenguajes del lado del servidor como Node.js, Go, PHP, Ruby, Python, Java y .NET. También emplean herramientas como MySQL, Oracle y SQL Server para buscar, guardar o modificar datos y enviarlos de vuelta al usuario a través del código front-end.

Por ejemplo, se prefiere Python por su legibilidad y sencillez, lo que lo convierte en una opción popular para aplicaciones web que requieren un desarrollo rápido. Java, por otro lado, es reconocido por su rendimiento y escalabilidad, ideal para aplicaciones empresariales a gran escala.

Las ofertas de empleo para desarrolladores back-end suelen requerir experiencia con frameworks de PHP como Laravel, Symfony y CakePHP; software de control de versiones como Git; y el uso de Linux como sistema de desarrollo y despliegue. El conocimiento de estas herramientas permite a los desarrolladores back-end crear aplicaciones web robustas, escalables y seguras.

Los desarrolladores back-end colaboran con los líderes e interesados del negocio (*stakeholders*) para comprender sus necesidades, traduciéndolas en requisitos técnicos y diseñando soluciones eficientes para estructurar la tecnología. Esto implica no solo escribir código, sino también diseñar la arquitectura general del sistema, garantizando que el sitio web pueda soportar un alto volumen de tráfico, procesar transacciones de forma segura y escalar según sea necesario.

Uno de los aspectos más críticos del desarrollo back-end es la gestión de bases de datos. Los desarrolladores deben diseñar y mantener bases de datos que almacenen la información de forma segura y la recuperen con rapidez. Esto a menudo implica trabajar con bases de datos relacionales como MySQL o PostgreSQL, o incluso con bases de datos NoSQL como MongoDB para modelos de datos más flexibles.

"Siempre he preferido el desarrollo back-end porque me encanta manipular datos", dice JP Toto, un desarrollador back-end con una larga trayectoria que trabaja actualmente en Wildbit. "Crear APIs que el público encuentre útiles es una parte muy satisfactoria de mi trabajo".

¿Qué es un desarrollador Full Stack?

A menudo no existe una distinción en blanco y negro entre el desarrollo front-end y el back-end. "Los desarrolladores front-end suelen necesitar aprender esas habilidades adicionales de back-end, y viceversa, especialmente en la economía actual donde los recursos de marketing son limitados", señaló Matranga. "Los desarrolladores necesitan algo de esa transversalidad disciplinaria. Muchas veces, tienes que ser un generalista".

Los desarrolladores full stack son todólogos (expertos en todo).

Aquí entra en escena el desarrollador full stack. Este rol fue popularizado por el departamento de ingeniería de Facebook. La idea es que un desarrollador full stack puede trabajar de forma multifuncional en todo el "stack" (pila) de tecnología, es decir, tanto en el front-end como en el back-end. Los desarrolladores full stack ofrecen el paquete completo.

"Trabajar profesionalmente tanto en el lado del servidor como en el del cliente abre más oportunidades", afirmó Federico Ulfo, desarrollador Full Stack en Grovo. Sin embargo, por supuesto, el desarrollo full stack no está exento de desafíos. "Para hacer una analogía con la comida, puedes ser bueno cocinando o bueno horneando, pero dominar ambos requiere tiempo y experiencia. Y no me refiero a seguir una receta, eso lo puede hacer cualquiera. Me refiero a tener los ingredientes para preparar algo realmente bueno".

Habilidades y herramientas necesarias para los desarrolladores Full Stack

Los desarrolladores full stack trabajan en el lado del servidor al igual que los desarrolladores back-end, pero también dominan los lenguajes de front-end que controlan cómo se ve el contenido en la parte del sitio web que está de cara al usuario. Su caja de herramientas incluye HTML, CSS, JavaScript y lenguajes del lado del servidor.

Un desarrollador full stack debe tener conocimientos en cada uno de los niveles sobre cómo funciona la web: desde configurar servidores Linux y escribir APIs del lado del servidor, hasta sumergirse en JavaScript del lado del cliente y aplicar un "ojo para el diseño" al CSS. Esta comprensión integral les permite construir aplicaciones web completas desde cero.

Por ejemplo, un desarrollador full stack podría comenzar configurando un servidor Linux para alojar una aplicación web y luego pasar a escribir la API que la aplicación utilizará para comunicarse con el servidor. Después, podría desarrollar la interfaz de

front-end, asegurándose de que sea adaptable (*responsive*) y fácil de usar para el usuario.

El desarrollo full stack requiere no solo experiencia técnica, sino también la capacidad de pensar estratégicamente sobre cómo interactúan los diferentes componentes de un sitio web. Los desarrolladores full stack deben ser capaces de ver el panorama general, entendiendo cómo los cambios en una parte del *stack* pueden afectar a toda la aplicación.

El rol de los CMS en el desarrollo Full-Stack

Para los desarrolladores full-stack, dominar los Sistemas de Gestión de Contenidos (CMS) es un componente clave de su conjunto de habilidades. Un CMS, como WordPress, Joomla o Drupal, agiliza el proceso de construcción y gestión de sitios web al proporcionar una plataforma robusta que se encarga de todo, desde la creación de contenidos hasta la gestión de usuarios. Al integrar un CMS, los desarrolladores full-stack pueden desarrollar de manera eficiente sitios web dinámicos que resultan fáciles de actualizar y mantener para usuarios sin conocimientos técnicos. Esto no solo acelera el tiempo de desarrollo, sino que también garantiza la escalabilidad y la seguridad. Los desarrolladores full-stack aprovechan su conocimiento tanto de las tecnologías front-end como back-end para personalizar y ampliar las funcionalidades de los CMS, creando soluciones a la medida que satisfacen necesidades comerciales específicas. Ya sea desarrollando complementos (*plugins*) personalizados o de optimizando las interacciones con las bases de datos, el uso de un CMS capacita a los desarrolladores full-stack para ofrecer soluciones web integrales de principio a fin.

Salario de un desarrollador Front-End frente al de un desarrollador Back-End

Al considerar una carrera en el desarrollo web, el salario suele ser un factor crítico. Para el año 2024, los desarrolladores front-end pueden esperar ganar un salario medio promedio de $96,000 dólares anuales en los EE. UU., dependiendo de su experiencia y ubicación. Por otro lado, los desarrolladores back-end suelen percibir

salarios más altos, con un salario medio promedio que arranca desde los $164,788 dólares al año debido a la complejidad técnica de su trabajo.

Los desarrolladores full stack, quienes combinan habilidades de ambas áreas, tienen un salario medio promedio de $137,085 dólares anuales en los Estados Unidos. La capacidad de trabajar tanto en el front-end como en el back-end hace que los desarrolladores full-stack sean particularmente valiosos, ya que pueden aportar en múltiples aspectos de un proyecto.

Los salarios también pueden variar significativamente según la industria y el tamaño de la empresa. Por ejemplo, los desarrolladores que trabajan en centros tecnológicos como Silicon Valley o la ciudad de Nueva York podrían exigir salarios más altos en comparación con aquellos en mercados más pequeños. Además, los desarrolladores con experiencia en tecnologías de vanguardia como la inteligencia artificial (IA), el aprendizaje automático (*machine learning*) o el *blockchain* pueden ver un potencial de ingresos aún mayor.

El desarrollo web tiene muchas facetas, cada una con su propio conjunto de responsabilidades, herramientas y habilidades. Ya sea que elijas especializarte en el desarrollo front-end, back-end o full-stack, recuerda que las habilidades blandas (*soft skills*) sólidas, como la atención al detalle, la resolución de problemas y la comunicación efectiva, te harán destacar.

La demanda de desarrolladores web calificados no muestra signos de desaceleración. Según la Oficina de Estadísticas Laborales de los EE. UU. (BLS), se espera que el empleo para desarrolladores web crezca entre un 16% y un 17% desde 2022 hasta 2032, mucho más rápido que el promedio de todas las ocupaciones.

Además de las habilidades técnicas, mantenerse actualizado con las últimas tendencias y aprender continuamente nuevas tecnologías es crucial. El campo del desarrollo web es dinámico, y regularmente surgen nuevos *frameworks*, librerías y herramientas. El aprendizaje de por vida y la adaptabilidad son esenciales para el éxito en esta industria en constante evolución.

En conclusión (o **El veredicto final**)

El desarrollo web tiene muchas facetas. Pero sin importar el tipo de desarrollo que estés pensando elegir, las habilidades blandas como la atención al detalle, la capacidad de aprender rápido, la habilidad para resolver problemas de manera

eficiente, la formación empresarial y una comunicación sólida te serán de gran utilidad, además de las habilidades técnicas descritas anteriormente.